Chapter 20 — Interview Question Bank

This chapter collects the Spring questions that come up again and again in real interviews, worded the way candidates report them on LeetCode Discuss interview-experience posts, Glassdoor reviews, public Notion and GitHub question lists, and the big "top N Spring Boot questions" pages. It is a **drill list**: one place to check that you have an answer ready for everything an interviewer is likely to ask about Spring Framework 7, Spring Boot 4.1 and Spring Security 7 on Java 25.

Chapter 18 is the book's *teaching* round of interview questions (168 answers with proofs, a project-walkthrough template and debugging scenarios). This bank is wider and shorter: it adds the questions Chapter 18 does not ask (Spring vs Spring Boot, stereotype annotations, `CommandLineRunner`, `CrudRepository` vs `JpaRepository`, the Hibernate caches, `get` vs `load`, isolation anomalies, `@ControllerAdvice` details, JWT filter code, saga, idempotency keys, rate limiting, coding tasks, scenario questions and more), and for questions that Chapter 18 already answers it gives a fresh short answer plus a link to the Chapter 18 entry and the teaching section.

How to use this bank

- **Wording.** Each question is written the way it is usually asked, so you recognise it when you hear it. Several common wordings of the same question are merged into one entry.
- **★ = asked very often.** These appeared in most of the lists and interview reports used for this bank. If you are short on time, do only the ★ questions first.
- **Answers.** Each answer is short enough to say out loud in 30–60 seconds: what it is, how it works, and the consequence that bites people. Code snippets are kept to ten lines.
- **Links.** "→ *More:*" points to the section of this book that teaches the topic in depth (with runnable code and real output), and to the matching Chapter 18 question when there is one. A question with no link is not covered elsewhere in the book — its answer here is complete.
- **How to drill.** Cover the answer, say yours out loud, then compare. Mark the ones you missed and come back to them the next day. Before an interview, re-read only the ★ questions and the scenario section.
- **Versions.** Answers are correct for Spring Boot 4.1.1, Spring Framework 7.0, Spring Security 7.1, Spring Data 2026.0, Hibernate ORM 7.4 and Java 25 (Boot needs Java 17 at minimum). When behaviour differs in Boot 3, the answer says so.

A few words used everywhere below:

- **Bean** — an object created and managed by Spring's container.
- **Container** (or **application context**) — the Spring object that creates beans, connects them and manages their life.
- **Proxy** — a generated stand-in object that wraps a bean and runs extra code (transactions, caching, security checks) before and after the real method.
- **Starter** — a Maven/Gradle dependency that pulls in everything needed for one feature (for example `spring-boot-starter-webmvc`).

Spring and Spring Boot fundamentals

Q1. ★ What is the Spring Framework, and why is it used? Spring is an open-source Java framework whose core is a **container** that creates your objects and wires them together (dependency injection). On top of that core it offers modules for web apps (Spring MVC, WebFlux), data access and transactions, security, messaging, testing and more. Teams use it because it removes boilerplate (repetitive setup code), keeps classes loosely coupled and testable, and gives one consistent programming model for everything a back-end service needs. → *More:* §1.1 What Spring is

Q2. ★ What is Spring Boot? What is the difference between Spring and Spring Boot? Spring Boot is an opinionated layer on top of the Spring Framework that makes an application production-ready with almost

no setup. Plain Spring gives you the building blocks but you must choose library versions, declare infrastructure beans and deploy to an external server. Boot adds **starters** (curated dependency sets), **auto-configuration** (beans created for you based on what is on the classpath), an **embedded server** (Tomcat runs inside your app, so you start it with `java -jar`), externalized configuration and Actuator for monitoring. Spring Boot is not a replacement for Spring — every Boot app *is* a Spring app.

Follow-up — What does "convention over configuration" mean in Spring Boot? The framework assumes sensible defaults so you only write configuration when you want something different. Examples: the server port is 8080, `application.properties` is read from the classpath, a `DataSource` (database connection pool) is created when a JDBC driver and a URL are present, and Jackson is configured for JSON. You override a convention with a property or your own bean; you never have to restate the default. → *More:* §1.2 What Spring Boot adds · §3.10 How auto-configuration works

Q3. ⋆ What are the main features and advantages of Spring Boot? Starters, auto-configuration, embedded servers (Tomcat by default, Jetty as an option; Undertow was removed in Boot 4), dependency management through a BOM (Bill of Materials — a list of library versions known to work together), externalized configuration with profiles, Actuator health and metrics endpoints, DevTools for fast restarts, and strong testing support (test slices, Testcontainers integration). The net effect: a working REST service in minutes, consistent versions, and sane production defaults that you can override one property at a time.

Follow-up — What are the key components of Spring Boot? Five pieces are usually expected: (1) **starters**, (2) **auto-configuration**, (3) the `SpringApplication` class that bootstraps the context and embedded server, (4) **externalized configuration** (`application.properties`/`.yaml`, environment variables, profiles), and (5) **Actuator** for production monitoring. Some lists add the Spring Boot CLI and Spring Initializr. → *More:* §1.2 What Spring Boot adds · §1.7 Starters · §1.8 The main class and @SpringBootApplication

Q4. How do you create a Spring Boot project? The usual way is **Spring Initializr** — the website start.spring.io (also built into IntelliJ IDEA, VS Code and the Spring CLI) — where you choose the build tool, Java version, Boot version and starters, then download a ready project. You can also write the `pom.xml` by hand with `spring-boot-starter-parent` as the parent. The generated project has a main class annotated with `@SpringBootApplication` and a test that loads the context. → *More:* §1.5 Creating a project with start.spring.io · §1.6 Project layout and the build file

Q5. ⋆ What happens when a Spring Boot application starts? (How does `SpringApplication.run()` work?) `SpringApplication.run()` (1) decides the application type (servlet, reactive or none) from the classpath, (2) prepares the `Environment` by loading property sources and profiles, (3) creates the `ApplicationContext`, (4) registers bean definitions from component scanning and auto-configuration, (5) refreshes the context — creating all singleton beans and starting the embedded web server, (6) runs `CommandLineRunner`/`ApplicationRunner` beans, and (7) publishes `ApplicationReadyEvent`. With `java -jar`, a small launcher class first sets up a class loader for the nested jars inside the "fat jar" and then calls your `main`. → *More:* §1.9 Running the application · Ch 18 Q22

Q6. What is an embedded server, and why is it useful? Can you replace Tomcat? An embedded server is a web server library (Tomcat by default) started inside your application's own JVM, instead of deploying a WAR file into a separately installed server. One artifact, one command, same runtime in every environment — ideal for containers. To switch to Jetty, exclude `spring-boot-starter-tomcat` and add `spring-boot-starter-jetty`. Boot 4 supports Tomcat 11 and Jetty 12.1; Undertow was dropped because it does not support Servlet 6.1.

Follow-up — Can you build a non-web Spring Boot application? Can you disable the web server? Yes. Leave out the web starter, or set `spring.main.web-application-type=none`. The context still starts, beans are created, and `CommandLineRunner` beans do the work — typical for batch jobs, CLI tools and Kafka consumers that need no HTTP endpoints. → *More:* §1.9 Running the application · §17.13 Spring Boot 3.x → 4.x migration table

Q7. What is the default port, and how do you change it? 8080. Change it with `server.port=9090` in `application.properties`, an environment variable `SERVER_PORT=9090`, or a command-line argument `--server.port=9090`. `server.port=0` picks a random free port (useful in tests); `server.port=-1` starts no HTTP listener at all. → *More:* §3.3 Precedence: which value wins?

Q8. ⋆ What are `CommandLineRunner` and `ApplicationRunner`? How do you run code at startup? Both are interfaces with a single `run` method that Boot calls once, after the context is fully started and

before `ApplicationReadyEvent`. `CommandLineRunner.run(String... args)` gets the raw arguments; `ApplicationRunner.run(ApplicationArguments args)` gets them parsed (options such as `--name=x` vs plain values). Order several runners with `@Order`. Other options: an `@EventListener(ApplicationReadyEvent.class)` method (runs when the app is ready to serve traffic) or `@PostConstruct` (runs while the bean is being created — too early for work that needs other beans to be fully ready).

```
@Bean
CommandLineRunner seed(ProductRepository repo) {
    return args -> repo.save(new Product("demo"));
}
```

Follow-up — How do you read command-line arguments or properties in code? Inject `ApplicationArguments` (parsed options and non-option args), use `@Value("${name}")`, or inject `Environment` and call `getProperty("name")`. Any `--key=value` argument becomes a property with the highest precedence, so normal `@Value`/`@ConfigurationProperties` binding sees it. → *More:* §1.9 Running the application · §2.7 The bean lifecycle · §3.1 Externalized configuration and the Environment

Q9. WAR vs executable JAR deployment — what is the difference? An executable ("fat") JAR contains your classes, all dependencies and the embedded server; you run it with `java -jar`. A WAR (Web Application Archive) is deployed into an external servlet container that is managed separately. Boot can still build a WAR (extend `SpringBootServletInitializer`, mark the Tomcat starter as `provided`), but JARs are the norm today because they fit containers and need no shared server. → *More:* §1.6 Project layout and the build file · §15.11 Faster startup: extracted jars, CDS, AOT cache, native images

Q10. What is the Spring Boot starter parent, and how does dependency management work? `spring-boot-starter-parent` is a Maven parent POM that imports the Boot BOM (`spring-boot-dependencies`), so you declare dependencies *without versions* and get a tested combination; it also sets the Java version, encoding, resource filtering and plugin defaults. If your project already has a parent, import `spring-boot-dependencies` in `<dependencyManagement>` instead. Override one version with a property such as `<jackson.version>`. → *More:* §1.6 Project layout and the build file

Q11. What is Spring Boot DevTools? Is it used in production? DevTools speeds up local development: automatic restart when classes change (using two class loaders so only your code reloads), and development-friendly defaults such as disabled template caching. Its LiveReload browser-refresh server is off by default in Boot 4 and deprecated since 4.1. It switches itself off when the app runs from a packaged jar (`java -jar`), and should be declared `optional`/`developmentOnly` so it never ships. → *More:* §1.10 DevTools · Ch 18 Q35

Q12. Which Java version does Spring Boot 4 need, and what are the headline changes in Boot 4? Java 17 at minimum (Java 25 recommended). Headlines: Spring Framework 7, modularized auto-configuration (one module and starter per technology), Jakarta EE 11 (Servlet 6.1, JPA 3.2), Jackson 3, JSpecify null-safety annotations, built-in API versioning, HTTP service client auto-configuration, `@MockBean` removed in favour of `@MockitoBean`, core `@Retryable`, Spring Security 7's lambda-only DSL, and Undertow removed. → *More:* §17.13 Spring Boot 3.x → 4.x migration table · Ch 18 Q166

IoC container, dependency injection, beans and scopes

Q13. ⋆ What is Inversion of Control (IoC)? What is Dependency Injection (DI)? How are they different? IoC is the general principle: your code no longer creates and controls the objects it depends on — a framework does, and calls into your code. DI is the specific technique Spring uses to achieve IoC: a class *declares* what it needs (usually as constructor parameters) and the container *injects* matching beans. IoC is the "what" (control moves to the framework); DI is the "how". The payoff is loose coupling: code depends on interfaces, and you can pass a fake in a unit test. → *More:* §2.1 Inversion of Control and Dependency Injection · Ch 18 Q1

Q14. What is the IoC container? What does it do? The IoC container is the `ApplicationContext`. It reads **configuration metadata** (annotations, `@Bean` methods, and rarely XML), builds a `BeanDefinition` for each bean (class, scope, dependencies, init/destroy methods), creates the beans in dependency order, injects them into each other, applies post-processors (which add proxies for `@Transactional`, `@Async` and so

on), and destroys them on shutdown. → *More:* §2.2 The container: BeanFactory vs ApplicationContext · §2.3 Defining beans

Q15. ⋆ `BeanFactory` vs `ApplicationContext`? `BeanFactory` is the basic container interface: create beans lazily, on the first `getBean` call. `ApplicationContext` extends it and adds what real applications need: eager creation of singletons at startup (so wiring errors fail fast), automatic registration of `BeanPostProcessor`s, event publishing, the `Environment` with profiles and properties, internationalized messages and resource loading. You always use an `ApplicationContext`; `BeanFactory` matters only for understanding. → *More:* §2.2 The container: BeanFactory vs ApplicationContext · Ch 18 Q3

Q16. What is a Spring bean? Why make an object a bean instead of calling `new`? A bean is an object whose creation, wiring and lifecycle the container manages. Being a bean lets Spring inject its dependencies, apply cross-cutting behaviour through proxies (`@Transactional`, `@Cacheable`, `@PreAuthorize` only work on beans), manage its scope and call its init/destroy callbacks. An object made with `new` gets none of this. Simple value objects (DTOs — data transfer objects, entities, records) should *not* be beans. → *More:* §2.3 Defining beans · Ch 18 Q2

Q17. ⋆ What are the types of dependency injection? Which one is recommended and why? Constructor, setter and field injection. Prefer **constructor injection**: fields can be `final`, the object is never half-built, the dependencies are visible in one place, unit tests can just call `new`, and circular dependencies fail at startup instead of hiding. Setter injection is for truly optional or changeable dependencies. Field injection (`@Autowired` on a private field) hides dependencies and needs reflection or Spring to test; keep it for test classes.

Follow-up — Is `@Autowired` required on a constructor? Not when the class has exactly one constructor — Spring uses it automatically (since Framework 4.3). With several constructors, annotate the one Spring should use. This is why a Lombok `@RequiredArgsConstructor` or a hand-written single constructor needs no annotation. → *More:* §2.4 Injection types · Ch 18 Q6

Q18. ⋆ What is the difference between `@Component`, `@Service`, `@Repository` and `@Controller`? All four are **stereotype annotations**: they mark a class for component scanning so it becomes a bean. `@Service` and `@Controller`/`@RestController` are meta-annotated with `@Component` and mainly document the layer (`@Controller` is also detected by Spring MVC as a request handler). `@Repository` has one real extra: a post-processor translates database exceptions thrown by that bean into Spring's `DataAccessException` hierarchy. Technically you could use `@Component` everywhere, but the specific ones make intent clear and allow pointcuts per layer. → *More:* §2.3 Defining beans · §7.5 Exception translation: DataAccessException

Q19. ⋆ `@Component` vs `@Bean`? `@Component` goes on *your own class* and the class is picked up by scanning. `@Bean` goes on a *method* in a `@Configuration` class; the method's return value becomes the bean. Use `@Bean` for third-party classes you cannot annotate (an `ObjectMapper`, a `RestClient`), or when creation needs logic or conditions. → *More:* §2.3 Defining beans · Ch 18 Q4

Q20. What does `@Configuration` do? What happens if you put `@Bean` methods in a plain `@Component`? `@Configuration` marks a class as a source of bean definitions. By default (`proxyBeanMethods = true`) Spring generates a CGLIB subclass so that when one `@Bean` method calls another, the call returns the existing singleton instead of building a new object. In a plain `@Component` (or with `proxyBeanMethods = false`, "lite mode") that call is a normal Java call and creates a second instance. Prefer passing dependencies as `@Bean` method parameters — then both modes behave the same. → *More:* §2.3 Defining beans · Ch 18 Q5

Q21. What is component scanning? What does `@ComponentScan` do? At startup Spring searches packages for classes annotated with stereotypes and registers them as beans. `@SpringBootApplication` includes `@ComponentScan` for the package of the main class and all sub-packages. That is why the main class belongs in the root package: a `@Service` in a sibling package outside it is silently not found, and you get "No qualifying bean". Use `@ComponentScan(basePackages = ...)` or `@Import` for code elsewhere. → *More:* §2.3 Defining beans · §1.8 The main class and @SpringBootApplication

Q22. ⋆ Two beans of the same type exist. How does Spring decide which one to inject? (`@Primary` vs `@Qualifier`) Spring matches by type. With several candidates it uses, in order: `@Qualifier("name")` at the injection point; a bean marked `@Primary`; the only bean not marked `@Fallback` (Framework 6.2+); a bean whose name equals the parameter name. Otherwise startup fails with `NoUniqueBeanDefinitionException`. `@Primary` is a default chosen by the bean's author; `@Qualifier` is a specific choice made by the consumer — so `@Qualifier` wins. → *More:* §2.5 How Spring picks a bean · Ch 18 Q7 · Ch 18 Q8

Q23. How do you inject all implementations of an interface? Inject a `List<PaymentGateway>` (ordered by `@Order` or `Ordered`) or a `Map<String, PaymentGateway>` keyed by bean name. This is the standard way to build a strategy pattern: the map lets you pick an implementation by a key at runtime without `if`/`switch` chains.

```
@Service
class PaymentRouter {
    private final Map<String, PaymentGateway> gateways;
    PaymentRouter(Map<String, PaymentGateway> gateways) { this.gateways = gateways; }
    PaymentGateway forType(String type) { return gateways.get(type + "Gateway"); }
}
```

→ *More:* §2.5 How Spring picks a bean

Q24. `@Autowired` vs `@Inject` vs `@Resource`? `@Autowired` is Spring's own annotation: resolves by type, has a `required` flag. `@Inject` (Jakarta Dependency Injection, `jakarta.inject`) behaves like `@Autowired` without `required`; pair it with `@Named` to qualify. `@Resource` (Jakarta Annotations) resolves **by name first**, then by type. In modern code, constructor injection without any annotation makes the choice mostly irrelevant. → *More:* §2.4 Injection types

Q25. How do you inject an optional dependency? Use `ObjectProvider<T>` (`getIfAvailable()`, `ifAvailable(...)`), `Optional<T>`, `@Autowired(required = false)` on a setter, or a nullable parameter annotated with JSpecify `@Nullable`. `ObjectProvider` is the most flexible: it is also lazy and can stream several candidates. → *More:* §2.4 Injection types · Ch 18 Q9

Q26. ⋆ What are the bean scopes? What is the default? Singleton (default — one instance per container), **prototype** (a new instance every time the bean is requested), and in web apps **request** (one per HTTP request), **session** (one per HTTP session), **application** (one per `ServletContext`) and **websocket**. Note: Spring's singleton is "one per container", not the GoF singleton pattern of "one per class loader". → *More:* §2.6 Bean scopes · Ch 18 Q10

Q27. Singleton vs prototype scope — when would you use prototype? A singleton is created once at startup and shared by every caller, so it must be stateless or thread-safe. A prototype is created fresh on every lookup and is **not destroyed by Spring** — the container hands it over and forgets it, so `@PreDestroy` never runs. Use prototype for objects that hold per-use mutable state (a builder, a non-thread-safe parser). In most services everything is a singleton. → *More:* §2.6 Bean scopes

Q28. ⋆ Are singleton beans thread-safe? No. Spring does nothing to make them thread-safe; it only guarantees one shared instance. Because one instance serves all concurrent requests, a singleton must be **stateless** — no mutable instance fields holding per-request data. If shared state is unavoidable, use thread-safe types (`ConcurrentHashMap`, `AtomicLong`), immutable objects, or move the state into a request-scoped bean or a method's local variables. → *More:* §2.6 Bean scopes

Q29. ⋆ What happens when you inject a prototype bean into a singleton? How do you fix it? The prototype is created once — when the singleton is built — and the singleton keeps that one instance forever, so it behaves like a singleton. Fixes: inject `ObjectProvider<MyProto>` and call `getObject()` each time; use a `@Lookup` method that Spring overrides to return a new instance; or use a scoped proxy (`@Scope(value = "prototype", proxyMode = ScopedProxyMode.TARGET_CLASS)`). → *More:* §2.6 Bean scopes · Ch 18 Q11 · §18.15 Proofs: five tricky answers, run

Q30. How can a singleton use a request-scoped or session-scoped bean? Inject a **scoped proxy**: declare the request-scoped bean with `proxyMode = ScopedProxyMode.TARGET_CLASS` (`@RequestScope` does this by default). The singleton receives a proxy; each method call on it finds the real instance bound to the current request. Calling it outside a request (in a scheduled job or another thread) throws `IllegalStateException` / "No thread-bound request found". → *More:* §2.6 Bean scopes · Ch 18 Q12

Q31. ⋆ Explain the Spring bean lifecycle.

1. Instantiate (constructor runs, constructor injection). 2) Populate properties (setter and field injection). 3) `Aware` callbacks (`BeanNameAware`, `ApplicationContextAware`). 4) `BeanPostProcessor.postProcessBeforeInitialization`. 5) Init callbacks — `@PostConstruct`, then `InitializingBean.afterPropertiesSet()`, then a custom `initMethod`. 6) `BeanPostProcessor.postProcessAfterInitialization` — **this is where AOP proxies are created**. 7) The bean is in use. 8) On shutdown, `@PreDestroy`, `DisposableBean.destroy()`, then `destroyMethod` (singletons only).

Follow-up — What are the `Aware` interfaces? Callback interfaces that let a bean receive a container object: `BeanNameAware` (its own bean name), `ApplicationContextAware` (the context), `EnvironmentAware`, `ResourceLoaderAware` and others. They are rarely needed in application code because you can simply inject `ApplicationContext` or `Environment` through the constructor. → *More:* §2.7 The bean lifecycle · Ch 18 Q13

Q32. `@PostConstruct` vs constructor vs `InitializingBean` — where should initialization code go? Put simple, self-contained setup in the constructor. Use `@PostConstruct` (from `jakarta.annotation`) when the code needs field/setter-injected values, or must run after injection but before the bean is used. `InitializingBean.afterPropertiesSet()` does the same but couples your class to Spring. Note: inside `@PostConstruct` the bean's own proxy does not exist yet, so a `@Transactional` method called from there runs without a transaction — use `ApplicationReadyEvent` for that. → *More:* §2.7 The bean lifecycle

Q33. `BeanPostProcessor` vs `BeanFactoryPostProcessor`? A `BeanFactoryPostProcessor` runs **before any bean is created** and can change bean *definitions* (for example, resolving `${...}` placeholders). A `BeanPostProcessor` runs **around the initialization of each bean instance** and can modify or wrap the instance — this is how `@Autowired` processing, `@Async` and AOP proxies work. Declare a `BeanFactoryPostProcessor` `@Bean` method as `static` so it doesn't force early creation of its configuration class. → *More:* §2.8 Post-processors: the container's extension points · Ch 18 Q14

Q34. ⋆ What is a circular dependency? How does Spring handle it, and how do you fix it? Bean A needs B and B needs A. With constructor injection Spring cannot create either and startup fails with `BeanCurrentlyInCreationException`. Since Boot 2.6 circular references are forbidden by default even for field/setter injection (`spring.main.allow-circular-references=false`). Real fixes: redesign (extract the shared logic into a third bean, or use events), or as a stop-gap make one side lazy (`@Lazy` on the parameter injects a proxy) or use `ObjectProvider`. → *More:* §2.9 Circular dependencies · Ch 18 Q15 · §18.15 Proofs: five tricky answers, run

Q35. What does `@Lazy` do? What does lazy initialization cost? `@Lazy` on a bean delays creating it until the first time it is needed; on an injection point it injects a proxy that resolves the real bean on first use. `spring.main.lazy-initialization=true` makes all beans lazy, which shortens startup but moves errors and latency to the first request, and hides misconfiguration until runtime. Use it selectively (heavy, rarely-used beans) and in development. → *More:* §2.10 Lazy initialization · Ch 18 Q16

Q36. What does `@DependsOn` do? It forces one bean to be initialized after the named beans, even when there is no injection between them (for example, a cache warmer that must run after a schema migration bean). On shutdown, the dependent bean is destroyed before the beans it depends on. Prefer a real dependency or events; `@DependsOn` is for hidden, side-effect ordering only.

Q37. How do application events work in Spring? Are they synchronous? A bean publishes an object with `ApplicationEventPublisher.publishEvent(...)`; methods annotated with `@EventListener` that accept that type are called. By default listeners run **synchronously on the publisher's thread** and inside its transaction, so a listener exception propagates to the publisher. Add `@Async` to run a listener on another thread, or use `@TransactionalEventListener` to run only after the transaction commits. → *More:* §2.12 Application events · Ch 18 Q18

Q38. How do you get the list of all beans in an application? Call `applicationContext.getBeanDefinitionNames()` (for example in a `CommandLineRunner`), or expose the Actuator `beans` endpoint (`/actuator/beans`), which lists each bean with its scope, type and dependencies. The `conditions` endpoint shows why auto-configured beans were or were not created. → *More:* §15.2 Spring Boot Actuator · §3.11 Seeing what Boot decided

Q39. (Framework 7) What are JSpecify null-safety annotations and `BeanRegistrar`? Framework 7 and Boot 4 annotate their APIs with JSpecify `@Nullable`/`@NullMarked` (a standard set of annotations that say whether a value may be null), so tools like NullAway and IDEs can catch null bugs at compile time. `BeanRegistrar` is a new interface for registering beans programmatically (for example, in a loop or based on properties) in a way that also works with AOT and native images; you `@Import` it from a configuration class. → *More:* §2.13 Null-safety with JSpecify (Framework 7) · §2.14 Programmatic registration with BeanRegistrar (Framework 7) · Ch 18 Q19 · Ch 18 Q20

Configuration, properties, profiles, auto-configuration and starters

Q40. ⋆ What does `@SpringBootApplication` do? Which three annotations does it combine? It combines `@SpringBootConfiguration` (a specialized `@Configuration`, so the main class can declare `@Bean`s), `@EnableAutoConfiguration` (turns on auto-configuration) and `@ComponentScan` (scans the main class's package and its sub-packages). Its `exclude` attribute disables chosen auto-configurations.

Follow-up — `@SpringBootApplication` vs `@EnableAutoConfiguration` — what is the difference? `@EnableAutoConfiguration` only turns on auto-configuration. `@SpringBootApplication` adds component scanning and marks the class as a configuration class. You would use `@EnableAutoConfiguration` alone only in unusual setups where you want to control scanning yourself. → *More:* §1.8 The main class and @SpringBootApplication · Ch 18 Q21

Q41. ⋆ What is auto-configuration, and how does it work internally? Auto-configuration is a set of `@AutoConfiguration` classes that Boot loads from the file `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` in each jar. Every class is guarded by **conditions** — `@ConditionalOnClass` (a library is on the classpath), `@ConditionalOnMissingBean` (you haven't defined your own), `@ConditionalOnProperty`, `@ConditionalOnWebApplication` and so on. They are processed after your own beans, so your definitions win and the auto-configured default "backs off". In Boot 4 these classes live in one module per technology (`spring-boot-webmvc`, `spring-boot-jdbc`, ...). → *More:* §3.10 How auto-configuration works · §3.14 Boot 4 modularization · Ch 18 Q24

Q42. `spring.factories` vs `AutoConfiguration.imports`? Before Boot 2.7, auto-configuration classes were listed in `META-INF/spring.factories` under the `EnableAutoConfiguration` key. Boot 2.7 introduced the dedicated `.imports` file, and since Boot 3.0 auto-configuration is read **only** from it. `spring.factories` is still used for other extension points (`EnvironmentPostProcessor`, `ApplicationListener`, failure analyzers). → *More:* §3.10 How auto-configuration works · §3.13 Writing your own auto-configuration and starter

Q43. How do you see which auto-configurations were applied, and why? Start with `--debug` (or `debug=true`) to print the **condition evaluation report**: "positive matches", "negative matches" with the failed condition, exclusions and unconditional classes. In a running app, Actuator's `/actuator/conditions` endpoint gives the same data as JSON. `/actuator/beans` shows what was created. → *More:* §3.11 Seeing what Boot decided · Ch 18 Q26

Q44. How do you disable or exclude a specific auto-configuration? `@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)`, or the property `spring.autoconfigure.exclude=<fully qualified class name>` (handy per profile). Often you don't need to exclude: just define your own bean of that type and `@ConditionalOnMissingBean` makes Boot's default back off. In Boot 4 remember that the class names moved to per-technology packages. → *More:* §3.12 Overriding and excluding auto-configuration · Ch 18 Q25

Q45. How does `@ConditionalOnMissingBean` work? What conditional annotations do you know? It registers the bean only if no bean of that type (or name) exists *at the time the condition is evaluated*. Because auto-configurations are processed after user configuration, it lets your bean replace the default. Its subtle trap: it only sees beans already registered, so it is reliable in auto-configuration classes but not in ordinary `@Configuration` classes. Others: `@ConditionalOnClass`/`OnMissingClass`, `@ConditionalOnBean`, `@ConditionalOnProperty` (with `havingValue`, `matchIfMissing`), `@ConditionalOnResource`, `@ConditionalOnWebApplication`, `@ConditionalOnExpression`, `@ConditionalOnThreading` (platform vs virtual threads). → *More:* §3.10 How auto-configuration works · §2.11 @Conditional and profiles

Q46. What is a starter? Name some common starters. A starter is a dependency-only artifact that pulls in a coherent set of libraries for one feature, plus the matching auto-configuration module. Common Boot 4 names: `spring-boot-starter-webmvc` (renamed from `-web`, which still works but is deprecated), `spring-boot-starter-webflux`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`, `spring-boot-starter-validation`, `spring-boot-starter-actuator`, `spring-boot-starter-aspectj` (formerly `-aop`), `spring-boot-starter-kafka`, `spring-boot-starter-test`. Boot 4 also adds a `-test` starter per technology (e.g. `spring-boot-starter-webmvc-test`). → *More:* §1.7 Starters · §3.14 Boot 4 modularization · Ch 18 Q23

Q47. ⋆ How do you create your own custom starter? Two modules by convention: an **autoconfigure** module containing an `@AutoConfiguration` class with conditions (`@ConditionalOnClass`, `@ConditionalOnMissingBean`) and an `@ConfigurationProperties` class, registered in `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports`; and a **starter** module that only

depends on it and on the libraries. Name it `acme-spring-boot-starter` (the `spring-boot-` prefix is reserved for Spring's own). Test it with `ApplicationContextRunner`. → *More:* §3.13 Writing your own auto-configuration and starter · Ch 18 Q33

Q48. What is the purpose of `application.properties` / `application.yml`? Which is better? They hold externalized configuration — values that change between environments without recompiling (URLs, credentials, timeouts, feature flags). Both are equivalent; YAML is hierarchical and supports several documents in one file (separated by `---`), properties files are flat and harder to mis-indent. If both exist in the same location, `.properties` wins for duplicate keys. Pick one per project. → *More:* §3.2 application.properties vs application.yaml

Q49. ⋆ In what order does Spring Boot load properties? How do you override a property? Higher wins (simplified, most important first): command-line arguments → Java system properties (`-D`) → OS environment variables → profile-specific files outside the jar → `application.properties` outside the jar → profile-specific files inside the jar → `application.properties` inside the jar → `@PropertySource` → defaults. Note the trap: any file *outside* the jar beats every file packaged *inside* it, even a profile-specific one. So you override any packaged value with an environment variable or a `--key=value` argument at deploy time. → *More:* §3.3 Precedence: which value wins? · Ch 18 Q27

Q50. How does an environment variable map to a property (relaxed binding)? Uppercase, replace `.` with `_`, drop `-`: `spring.datasource.url` ↔ `SPRING_DATASOURCE_URL`, `my-app.retry-count` ↔ `MYAPP_RETRYCOUNT`. A list index becomes its own underscore-separated segment: `my-app.hosts[0]` ↔ `MYAPP_HOSTS_0`, and `my.service[0].other` ↔ `MY_SERVICE_0_OTHER`. Relaxed binding also accepts `retryCount`, `retry-count` and `retry_count` in files; the kebab-case form is canonical. → *More:* §3.4 Environment variables and relaxed binding · Ch 18 Q28

Q51. ⋆ `@Value` vs `@ConfigurationProperties`? `@Value("${app.timeout:5s}")` injects one value, supports SpEL (Spring Expression Language) and is fine for one or two settings. `@ConfigurationProperties(prefix = "app")` binds a whole group of properties to a typed object (ideally a Java record): relaxed binding, nested objects, lists, `Duration`/`DataSize` conversion, validation with `@Validated`, and IDE auto-completion via generated metadata. For more than a couple of related settings, use `@ConfigurationProperties`.

```
@ConfigurationProperties("app.payment")
@Validated
record PaymentProps(@NotBlank String url, Duration timeout, int retries) {}
```

Follow-up — How do you make `@ConfigurationProperties` classes available? What does `@EnableConfigurationProperties` do? Register them with `@ConfigurationPropertiesScan` on the main class (scans like component scanning) or `@EnableConfigurationProperties(PaymentProps.class)` on a configuration class (typical in auto-configurations). Marking the class `@Component` also works but is discouraged. Records and single-constructor classes use constructor binding automatically. → *More:* §3.5 @Value · §3.6 @ConfigurationProperties — type-safe configuration · Ch 18 Q29

Q52. How do you make the application fail fast on invalid or missing configuration? Put `@Validated` on the `@ConfigurationProperties` class and Bean Validation constraints (`@NotBlank`, `@Min`, `@NotNull`) on its fields — binding fails at startup with a clear message. Avoid defaults that hide a missing secret (`${DB_PASSWORD:}`). Spring's `FailureAnalyzer`s turn common startup errors into readable reports. → *More:* §3.6 @ConfigurationProperties — type-safe configuration · Ch 18 Q30

Q53. ⋆ What are profiles? How do you activate one? A profile is a named set of beans and configuration that is active only in some environments (`dev`, `test`, `prod`). Profile-specific files are `application-{profile}.properties`. Activate with `spring.profiles.active=prod` (property, `SPRING_PROFILES_ACTIVE` env var, or `--spring.profiles.active=prod`), or `@ActiveProfiles` in tests. Beans can be restricted with `@Profile("dev")` or `@Profile("!prod")`. Profile groups (`spring.profiles.group.prod=proddb,prodmq`) activate several at once. → *More:* §3.7 Profiles, profile-specific files and profile groups · Ch 18 Q31

Q54. `@Profile` vs `@Conditional`? `@Profile` is a special, simple condition: "register this bean only if these profiles are active". `@Conditional` (and Boot's `@ConditionalOn...` family) can test anything — a class on the classpath, a property value, another bean. Prefer property-based conditions for feature switches and keep profiles for environment differences. → *More:* §2.11 @Conditional and profiles · Ch 18 Q17

Q55. How do you keep secrets (passwords, API keys) out of `application.properties`? Never commit them. Inject them at runtime through environment variables, Kubernetes Secrets mounted as files and imported with `spring.config.import=configtree:/run/secrets/`, or a secret store (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault) via the matching Spring Cloud integration. Rotate them and avoid logging them; Actuator masks values of sensitive keys by default. → *More:* §3.8 Importing configuration: spring.config.import · §11.13 Storing keys and secrets · Ch 18 Q32

Q56. What is `bootstrap.properties`, and is it still used? In older Spring Cloud versions `bootstrap.properties` was loaded by a parent "bootstrap context" before `application.properties`, mainly to locate the Config Server. Since Spring Cloud 2020 it is disabled by default; you use `spring.config.import=configserver:` in the normal `application.properties` instead. You can re-enable the legacy mechanism with `spring-cloud-starter-bootstrap`, but new projects should not. → *More:* §16.3 Centralized configuration: Spring Cloud Config

Q57. How do you enable HTTPS in a Spring Boot application? Configure an **SSL bundle** — a named set of key and trust material — and point the server at it: `spring.ssl.bundle.pem.web.keystore.certificate=...`, `...private-key=...`, then `server.ssl.bundle=web` and `server.port=8443`. Bundles can be reused for clients (`RestClient`, Kafka, Redis), and with `reload-on-update=true` the server picks up renewed certificate files without a restart. In Kubernetes, TLS is often terminated at the ingress instead. → *More:* §11.13 Storing keys and secrets

Q58. How do you configure logging in Spring Boot? How do you enable debug logging? Boot uses SLF4J (a logging facade) with Logback by default. Set levels with `logging.level.root=INFO` and `logging.level.com.acme=DEBUG`; write to a file with `logging.file.name`; switch to JSON with `logging.structured.format.console=ecs` (or `logstash`, `gelf`). `--debug` turns on debug logs for core loggers and prints the condition report. Levels can be changed at runtime through `/actuator/loggers`. → *More:* §15.8 Structured logging

---

AOP and proxies

Q59. ⋆ What is AOP? Explain aspect, advice, pointcut, join point and weaving. AOP (aspect-oriented programming) moves **cross-cutting concerns** — logic needed in many places such as logging, transactions, security and metrics — out of business code. An **aspect** is the module holding that logic. A **join point** is a point in execution where it can run (in Spring AOP: a method call on a bean). A **pointcut** is an expression selecting join points (`execution(* com.acme.service..*(..))`). **Advice** is the code that runs there (`@Before`, `@After`, `@AfterReturning`, `@AfterThrowing`, `@Around`). **Weaving** is linking aspects into the target — Spring does it at runtime with proxies. → *More:* §4.1 Cross-cutting concerns and the AOP vocabulary · Ch 18 Q36

Q60. How does Spring AOP work internally? After a bean is initialized, a `BeanPostProcessor` checks whether any advisor's pointcut matches its methods. If so, it returns a **proxy** instead of the original object; callers get the proxy injected. A call goes proxy → chain of interceptors (advice) → real method, and back. Only calls that come through the proxy are advised. A JDK proxy can intercept only interface methods; a CGLIB proxy can intercept any method it can override — never `private`, `final` or `static` ones. Keep advised methods public: some features (caching, for example) honour only public methods, although `@Transactional` also works on protected and package-private methods with class-based proxies since Framework 6.0. → *More:* §4.4 How Spring AOP works: proxies · Ch 18 Q37

Q61. ⋆ JDK dynamic proxy vs CGLIB proxy? A JDK dynamic proxy implements the bean's **interfaces** — callers must use the interface type. CGLIB creates a **subclass** of the bean class at runtime, so it works without interfaces but cannot intercept `final` classes/methods or `private` methods. Plain Spring Framework uses a JDK proxy when the bean implements an interface; Spring Boot defaults to CGLIB for everything (`spring.aop.proxy-target-class=true`), which avoids "bean is not of required type" errors when injecting by class. → *More:* §4.4 How Spring AOP works: proxies · Ch 18 Q38

Q62. ⋆ What is the self-invocation problem? How do you solve it? When a method calls another method of the *same* bean (`this.save()`), the call doesn't pass through the proxy, so the second method's `@Transactional`, `@Async`, `@Cacheable` or `@Retryable` is ignored. Fixes, best first: move the method into another bean; inject the bean's own proxy (`@Lazy` self-injection or `ObjectProvider<Self>`) and call

through it; use `TransactionTemplate` programmatically; or switch to AspectJ weaving. → *More:* §4.5 Proxy limitations — proved · Ch 18 Q39 · §18.15 Proofs: five tricky answers, run

Q63. Which Spring features are built on proxies? `@Transactional`, `@Cacheable`/`@CacheEvict`, `@Async`, `@PreAuthorize`/`@PostAuthorize`, `@Validated` method validation, Framework 7 `@Retryable` and `@ConcurrencyLimit`, `@Observed`, scoped beans (request/session proxies), `@Lazy` injection points, full `@Configuration` classes (CGLIB) and Spring Data repositories (interfaces implemented at runtime). They all share the same limits: self-invocation, `final`/`private` methods, and objects created with `new`. → *More:* §4.6 Spring features built on proxies · Ch 18 Q40

Q64. Write an aspect that logs the execution time of methods annotated with a custom annotation.

```
@Retention(RUNTIME) @Target(METHOD) public @interface Timed {}

@Aspect @Component
class TimingAspect {
    @Around("@annotation(com.acme.Timed)")
    Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.nanoTime();
        try { return pjp.proceed(); }
        finally { log.info("{} took {} ms", pjp.getSignature(), (System.nanoTime() - start) / 1_000_000); }
    }
}
```

`@Around` must call `proceed()` and **return its result**, otherwise the caller gets `null`. Needs `spring-boot-starter-aspectj` in Boot 4. → *More:* §4.2 Writing aspects with @Aspect · §4.3 Pointcut expressions

Q65. `@Around` vs the other advice types? How do you order two aspects? `@Around` wraps the call: it can change arguments, skip the call, retry it, change the return value or translate exceptions. The others are narrower and safer: `@Before`, `@AfterReturning` (sees the result), `@AfterThrowing` (sees the exception), `@After` (finally). Reach for `@Around` only when a narrower type can't do the job: with the others you can't forget to call `proceed()`. Order aspects with `@Order` — lower value = higher priority = outermost wrapper. → *More:* §4.2 Writing aspects with @Aspect · Ch 18 Q41 · Ch 18 Q42

Q66. Spring AOP vs AspectJ? Spring AOP: proxy-based, runtime, method calls on Spring beans only, zero build changes. AspectJ: a full AOP language that weaves bytecode at compile time or load time, so it can advise field access, constructors, private methods, self-invocation and non-bean objects. Spring AOP borrows AspectJ's annotations and pointcut syntax. Use AspectJ only when proxy limits really hurt. → *More:* §4.8 Spring AOP vs AspectJ · Ch 18 Q43

Q67. (Framework 7) What are `@Retryable` and `@ConcurrencyLimit` in Spring Framework 7? Framework 7 moved basic resilience into the core: `@Retryable` retries a method on failure with configurable `maxRetries` (default 3, so up to 4 calls in total), `delay`, `multiplier`, `maxDelay` and `jitter` (also for reactive return types); `@ConcurrencyLimit` caps how many threads may run a method at once (a simple bulkhead). Enable them with `@EnableResilientMethods`. They replace most uses of the separate Spring Retry project; for circuit breakers you still need Resilience4j. → *More:* §4.7 Framework 7 core resilience: @Retryable and @ConcurrencyLimit · Ch 18 Q44

Spring MVC, REST and HTTP clients

Q68. ⋆ What is the `DispatcherServlet`? Walk through how a request is processed in Spring MVC. The `DispatcherServlet` is the **front controller**: one servlet that receives every request and delegates. Flow: servlet filters (including Spring Security) → `DispatcherServlet` → `HandlerMapping` finds the controller method for the URL and HTTP method → interceptors' `preHandle` → `HandlerAdapter` resolves arguments (`@PathVariable`, `@RequestBody` via an `HttpMessageConverter` such as Jackson) and calls the method → the return value is written as JSON by a message converter (REST) or resolved to a view by a `ViewResolver` (server-side HTML) → `postHandle`/`afterCompletion`. Exceptions go to `HandlerExceptionResolver`s, which call your `@ExceptionHandler` methods. → *More:* §5.2 How a request flows: the DispatcherServlet · Ch 18 Q45

Q69. ⋆ `@Controller` vs `@RestController`? `@RestController` = `@Controller` + `@ResponseBody` on every method: return values are serialized into the response body (JSON by default). A plain `@Controller` returns view

names that a template engine (Thymeleaf) renders into HTML, unless a method has `@ResponseBody` or returns `ResponseEntity`. → *More:* §5.3 Controllers and request mappings · Ch 18 Q46

Q70. `@RequestMapping` vs `@GetMapping` (and `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`)? `@RequestMapping` is the general annotation: path, HTTP method, `consumes`, `produces`, headers, params. On a class it sets a common path prefix. `@GetMapping("/x")` is shorthand for `@RequestMapping(path = "/x", method = GET)`; the others likewise. Use the shortcuts on methods for readability. → *More:* §5.3 Controllers and request mappings

Q71. ★ `@PathVariable` vs `@RequestParam` vs `@RequestBody` (vs `@RequestHeader`, `@ModelAttribute`)? `@PathVariable` reads a segment of the URL path (`/orders/{id}`) — identifies a resource. `@RequestParam` reads a query-string or form parameter (`?page=2`) — filters, options; can be optional with a default. `@RequestBody` deserializes the whole request body (JSON) into an object via a message converter. `@RequestHeader` reads a header. `@ModelAttribute` binds several query/form parameters onto an object's fields (used for search forms or HTML forms), unlike `@RequestBody`, which reads JSON. → *More:* §5.4 Reading input and writing output · Ch 18 Q47

Q72. Why return `ResponseEntity`? How do you set status codes and headers? `ResponseEntity<T>` lets you control the status code, headers and body together: `ResponseEntity.created(location).body(dto)` for 201 with a `Location` header, `ResponseEntity.noContent().build()` for 204, `ResponseEntity.notFound().build()`. For a fixed status you can also use `@ResponseStatus(HttpStatus.CREATED)` on the method. Throw `ResponseStatusException` (or let an exception handler map exceptions) for error codes. → *More:* §5.4 Reading input and writing output · Ch 18 Q48

Q73. ★ What makes an API RESTful? Explain HTTP methods, status codes and idempotency. REST models the API as **resources** identified by URLs, manipulated with standard HTTP methods, with stateless requests and representations such as JSON. GET reads (safe — no side effects), POST creates or triggers an action, PUT replaces a resource, PATCH partially updates, DELETE removes. **Idempotent** means repeating the same request leaves the same state: GET, PUT and DELETE are idempotent; POST is not; PATCH may or may not be. Key status codes: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Content, 429 Too Many Requests, 500, 503. → *More:* §5.1 REST in one page · Ch 18 Q56

Q74. How do you design a REST endpoint for creating an order? Show a controller. Use a request DTO validated with `@Valid`, delegate to a service, return 201 with the new resource's URL. Never expose JPA entities directly.

```
@RestController @RequestMapping("/api/orders")
class OrderController {
    private final OrderService service;
    OrderController(OrderService service) { this.service = service; }

    @PostMapping
    ResponseEntity<OrderDto> create(@Valid @RequestBody CreateOrder req) {
        OrderDto saved = service.create(req);
        return ResponseEntity.created(URI.create("/api/orders/" + saved.id())).body(saved);
    }
}
```

→ *More:* §5.3 Controllers and request mappings · §5.13 Putting it together: design checklist and mistakes

Q75. Why should you not expose JPA entities directly in REST responses? Entities mirror the database, not the API contract: renaming a column breaks clients; lazy associations cause `LazyInitializationException` or extra queries during JSON serialization; bidirectional relationships cause infinite recursion; and you may leak fields like password hashes or allow **mass assignment** (clients setting fields they shouldn't, such as `role`). Map to DTOs — Java records are ideal — by hand or with a mapper such as MapStruct.

Follow-up — How do you handle a bidirectional relationship that causes infinite recursion in JSON? Best: return DTOs that include only one direction. Otherwise use Jackson's `@JsonManagedReference` on the parent side and `@JsonBackReference` on the child, or `@JsonIgnore` on one side, or `@JsonIdentityInfo` to write references by id. → *More:* §5.13 Putting it together: design checklist and mistakes · §8.12 Projections · §5.5 JSON with Jackson 3

Q76. How does Spring convert Java objects to JSON? How do you customize it? (Jackson 3 in Boot 4) An `HttpMessageConverter` backed by Jackson serializes return values and deserializes `@RequestBody`. Boot auto-configures the mapper; customize with `spring.jackson.*` properties, a `JsonMapperBuilderCustomizer` bean, or annotations such as `@JsonProperty`, `@JsonIgnore`, `@JsonFormat`. Boot 4 uses **Jackson 3**: packages are `tools.jackson.*` (annotations stay in `com.fasterxml.jackson.annotation`), the main type is `JsonMapper`, and dates are written as ISO strings by default. Jackson 2 support remains, deprecated, for migration. → *More:* §5.5 JSON with Jackson 3 · Ch 18 Q51

Q77. What is content negotiation? The server chooses the response format from the client's `Accept` header (for example `application/json` vs `application/xml`), and the request format from `Content-Type`. Spring picks the first `HttpMessageConverter` that can write the return type in an acceptable media type; `produces` / `consumes` on a mapping narrow it. If nothing matches, the response is 406 Not Acceptable (or 415 Unsupported Media Type for request bodies). → *More:* §5.8 Content negotiation · Ch 18 Q52

Q78. How do you version a REST API? (Framework 7) Common strategies: URL path (`/v1/orders`), a header (`X-API-Version` or `API-Version`), a query parameter, or the media type (`Accept: application/vnd.acme.v2+json`). Framework 7 adds **first-class API versioning**: configure the strategy once (`spring.mvc.apiversion.use.header=API-Version`, or `ApiVersionConfigurer`), then write `@GetMapping(path = "/orders", version = "2")`; Spring resolves the version, validates it, can mark versions deprecated, and clients (`RestClient`, HTTP interfaces) can send versions too. → *More:* §5.9 API versioning (Framework 7) · Ch 18 Q53

Q79. How do you implement pagination and sorting in a REST API? Accept a `Pageable` parameter in the controller (`?page=0&size=20&sort=createdAt,desc`) and pass it to a Spring Data repository method returning `Page<T>` or `Slice<T>`. Return a stable JSON shape — your own DTO, or Spring Data's `PagedModel` (content plus a `page` block with size, number, totalElements, totalPages) — rather than serializing the `PageImpl` class directly, whose JSON is not a stable contract (Spring Data warns about it). Cap the page size (`spring.data.web.pageable.max-page-size`). For deep pages use keyset (cursor) pagination. → *More:* §8.11 Pagination and sorting · §5.12 Documenting the API: OpenAPI with springdoc · Ch 18 Q57

Q80. ⋆ What is CORS? How do you configure it in Spring Boot? CORS (Cross-Origin Resource Sharing) is a browser rule: JavaScript on `https://app.example.com` may call `https://api.example.com` only if the API answers with `Access-Control-Allow-Origin` headers allowing it; for non-simple requests the browser first sends an OPTIONS "preflight". Configure globally with `WebMvcConfigurer.addCorsMappings`, per controller with `@CrossOrigin`, and — when Spring Security is used — enable `http.cors(...)` with a `CorsConfigurationSource` so preflights are not rejected before reaching MVC. Never combine `allowedOrigins("*")` with credentials. → *More:* §5.10 CORS and file upload · §10.9 CORS · Ch 18 Q54

Q81. ⋆ Filter vs interceptor vs AOP aspect — when to use which? A **servlet filter** runs before the `DispatcherServlet`, for every request, and sees raw request/response — use for security, CORS, request logging, compression, correlation ids. A **`HandlerInterceptor`** runs inside Spring MVC around controller methods and knows which handler was chosen — use for per-endpoint checks, locale, audit. An **aspect** wraps any bean method (not only web) — use for service-level concerns like timing or retries. → *More:* §5.11 Filters vs interceptors (vs AOP) · Ch 18 Q55

Q82. How do you upload and download files in Spring MVC? Upload: accept `@RequestParam("file") MultipartFile file` in a `multipart/form-data` endpoint; set limits with `spring.servlet.multipart.max-file-size` and `max-request-size`; validate content type and size; stream large files to storage instead of reading them into memory. Download: return `ResponseEntity<Resource>` with `Content-Disposition: attachment; filename=...` and the right content type, or `StreamingResponseBody` for big content. → *More:* §5.10 CORS and file upload

Q83. How do you document a REST API (Swagger/OpenAPI)? Add springdoc-openapi (version 3.x for Boot 4; not managed by the Boot BOM, so set its version). It inspects your controllers and generates an OpenAPI 3 document at `/v3/api-docs` and a Swagger UI at `/swagger-ui.html`. Refine with annotations such as `@Operation` and `@Schema`. Disable or secure the UI in production. → *More:* §5.12 Documenting the API: OpenAPI with springdoc

Q84. What is HATEOAS? Is it used in practice? HATEOAS (Hypermedia as the Engine of Application State) means responses include links telling the client what it can do next (`"_links": {"cancel": {...}}`). Spring HATEOAS provides `EntityModel`, `CollectionModel` and link builders, and Spring Data REST uses it.

In practice most internal JSON APIs skip it; it is worth knowing as the "fully RESTful" level of the Richardson maturity model. → *More:* §9.3 Spring Data REST

Q85. How do you return streaming or real-time data from Spring MVC (SSE, WebSocket)? For server-to-client streams, return an `SseEmitter` (Server-Sent Events — a long HTTP response carrying a stream of events) from MVC, or a `Flux<ServerSentEvent<T>>` from WebFlux. For two-way messaging, use WebSocket (`spring-boot-starter-websocket`), often with STOMP and a message broker. For a large download, `StreamingResponseBody` writes on a separate thread. → *More:* §13.12 Full-stack WebFlux tests and Server-Sent Events

Q86. ⋆ `RestTemplate` vs `WebClient` vs `RestClient` — which should you use? `RestClient` (Framework 6.1+) is the modern **synchronous** client with a fluent API; use it in Spring MVC apps — with virtual threads, blocking calls are cheap. `WebClient` is the **non-blocking** reactive client; use it in WebFlux apps or when you need streaming or many concurrent calls composed reactively. `RestTemplate` is the old template-style synchronous client; it is in maintenance, with formal deprecation planned for Framework 7.1 and removal in 8.0. For declarative clients, define an **HTTP service interface** (`@GetExchange`) backed by `RestClient`. → *More:* §6.1 Choosing a client · §6.9 RestTemplate and WebClient in brief · Ch 18 Q58

Q87. What are HTTP service interfaces (`@HttpExchange`)? How do they compare to Feign? You write a Java interface with `@GetExchange("/users/{id}") User get(@PathVariable long id);` and Spring generates the implementation on top of `RestClient` or `WebClient`. Boot 4 auto-configures them through `@ImportHttpServices` and `spring.http.serviceclient.<group>.*` properties (base URL, timeouts). They replace most uses of OpenFeign, which is now in maintenance mode. → *More:* §6.6 HTTP service interfaces (@HttpExchange) · §16.6 Declarative clients: OpenFeign vs HTTP interface clients · Ch 18 Q59 · Ch 18 Q160

Q88. How do you set timeouts and handle errors on an HTTP client? Always set a connect timeout and a read timeout (Boot 4: `spring.http.clients.connect-timeout`, `spring.http.clients.read-timeout`, or on the request factory); a missing timeout lets a slow dependency exhaust your threads. Handle error statuses with `.onStatus(HttpStatusCode::is4xxClientError, ...)` on `RestClient`, which otherwise throws `RestClientResponseException` subclasses. Retry only idempotent calls, with backoff, and wrap critical dependencies in a circuit breaker. → *More:* §6.3 Error handling · §6.4 Timeouts and the underlying HTTP library · Ch 18 Q60

Q89. How do you add a header (for example a correlation id or token) to every outgoing request? Register a `ClientHttpRequestInterceptor` on the `RestClient.Builder` (or an `ExchangeFilterFunction` on `WebClient`). Boot's auto-configured builders already propagate tracing headers; for OAuth2 tokens use Spring Security's `OAuth2ClientHttpRequestInterceptor`. → *More:* §6.5 Interceptors

---

Validation and exception handling

Q90. ⋆ How do you validate request data in Spring Boot? Add `spring-boot-starter-validation` (Hibernate Validator, the Bean Validation implementation). Put constraints on the DTO — `@NotBlank`, `@Size`, `@Email`, `@Positive`, `@Past`, `@Pattern` — and `@Valid` on the `@RequestBody` parameter. Invalid input throws `MethodArgumentNotValidException`, which Spring turns into 400 Bad Request. For path/query parameters, put constraints directly on the parameters; Framework 6.1+ validates them natively and throws `HandlerMethodValidationException`. → *More:* §5.6 Validation · Ch 18 Q49

Q91. `@Valid` vs `@Validated`? `@Valid` (Jakarta) triggers validation of an argument or a nested object (cascading). `@Validated` (Spring) additionally supports **validation groups** (validating a different set of constraints on create vs update) and, on a class, enables **method validation** through a proxy, so service methods can validate their parameters and return values. Use `@Valid` on nested fields to cascade into them. → *More:* §5.6 Validation

Q92. How do you write a custom validation annotation? Create an annotation marked `@Constraint(validatedBy = ...)` with `message`, `groups` and `payload` attributes, and a `ConstraintValidator` implementation with an `isValid` method. Validators are created by Spring, so they can inject beans (for example to check uniqueness — but beware that this hits the database on every validation).

```java
@Constraint(validatedBy = SkuValidator.class)
@Target(FIELD) @Retention(RUNTIME)
public @interface ValidSku { String message() default "invalid SKU";
    Class<?>[] groups() default {}; Class<? extends Payload>[] payload() default {}; }

class SkuValidator implements ConstraintValidator<ValidSku, String> {
    public boolean isValid(String v, ConstraintValidatorContext c) { return v == null || v.matches("[A-Z]{3}-
\\d{4}"); }
}
```

→ *More:* §5.6 Validation

Q93. How do you validate across several fields (for example, start date before end date)? Use a **class-level** constraint: annotate the DTO (not a field) with a custom constraint whose validator receives the whole object and compares the fields. Report the error on a specific field with `context.buildConstraintViolationWithTemplate(...).addPropertyNode("endDate")`. → *More:* §5.6 Validation

Q94. ⋆ How do you handle exceptions globally in Spring Boot? (`@ControllerAdvice` / `@RestControllerAdvice`) Create a class annotated with `@RestControllerAdvice` with `@ExceptionHandler` methods; each maps an exception type to a response for all controllers. Return RFC 9457 **Problem Details** (`ProblemDetail`: type, title, status, detail, instance, plus custom properties) for a consistent error format. Extending `ResponseEntityExceptionHandler` gives ready handlers for Spring MVC's own exceptions.

```java
@RestControllerAdvice
class ApiErrors {
    @ExceptionHandler(OrderNotFoundException.class)
    ProblemDetail notFound(OrderNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setTitle("Order not found");
        return pd;
    }
}
```

Follow-up — `@ControllerAdvice` vs `@RestControllerAdvice`? Same relationship as `@Controller` vs `@RestController`: `@RestControllerAdvice` = `@ControllerAdvice` + `@ResponseBody`, so handler return values are written as the response body (JSON). With plain `@ControllerAdvice`, a handler returns a view name unless it returns `ResponseEntity`/`ProblemDetail` or carries `@ResponseBody`. Both can be limited with `basePackages` or `assignableTypes`. → *More:* §5.7 Error handling and Problem Details (RFC 9457) · Ch 18 Q50

Q95. What are the ways to handle exceptions in Spring MVC, and which wins?

1. `@ExceptionHandler` inside the controller (local, wins first); 2) `@ExceptionHandler` in a `@ControllerAdvice` (global; order several advices with `@Order`); 3) `@ResponseStatus` on the exception class; 4) throwing `ResponseStatusException` or an `ErrorResponseException`; 5) a custom `HandlerExceptionResolver`; 6) Boot's fallback `/error` endpoint (`BasicErrorController`). Within one advice, the most specific exception type wins. Exceptions thrown in **filters** (including Spring Security) never reach `@ControllerAdvice`. → *More:* §5.7 Error handling and Problem Details (RFC 9457)

Q96. What does Spring Boot return for an unhandled error? How do you customize the default error response? The `/error` endpoint (`BasicErrorController`) returns JSON with `timestamp`, `status`, `error` and `path` — or a "Whitelabel Error Page" for browsers. With `spring.mvc.problemdetails.enabled=true` Spring MVC's own exceptions become `application/problem+json`. Customize with your own `@ControllerAdvice`, a custom `ErrorAttributes` bean, or `server.error.include-message`/`include-stacktrace` (keep stack traces off in production). → *More:* §5.7 Error handling and Problem Details (RFC 9457)

Q97. Should you use checked or unchecked exceptions in a Spring service? Mostly unchecked (`RuntimeException` subclasses such as `OrderNotFoundException`): they don't pollute signatures, and by default they trigger transaction rollback. A checked exception thrown from a `@Transactional` method **commits** unless you add `rollbackFor`. Spring itself translates checked JDBC `SQLException`s into unchecked `DataAccessException`s for the same reason. → *More:* §7.10 Rollback rules · §7.5 Exception translation: DataAccessException

---

Spring Data JPA and Hibernate

Q98. ⋆ JPA vs Hibernate vs Spring Data JPA? JPA (Jakarta Persistence) is a specification — annotations (`@Entity`, `@Id`) and interfaces (`EntityManager`) for object-relational mapping (ORM: mapping Java objects to database tables). **Hibernate** is the most common implementation of JPA (Boot 4.1 uses Hibernate ORM 7.4). **Spring Data JPA** sits on top: it generates repository implementations from interfaces, derives queries from method names, adds paging, auditing and Spring transaction integration. → *More:* §8.1 JPA, Hibernate and Spring Data — who does what · Ch 18 Q76

Q99. ⋆ `CrudRepository` vs `PagingAndSortingRepository` vs `JpaRepository`? `CrudRepository` gives basic CRUD (create, read, update, delete): `save`, `findById`, `findAll`, `deleteById`, `count`. `ListCrudRepository` returns `List` instead of `Iterable`. `PagingAndSortingRepository` adds `findAll(Pageable)` and `findAll(Sort)`. `JpaRepository` extends both list variants and adds JPA-specific methods: `flush`, `saveAndFlush`, `saveAllAndFlush`, `deleteAllInBatch`, `getReferenceById`. Use `JpaRepository` for JPA apps, or `Repository` plus only the methods you want to expose. → *More:* §8.8 Repositories

Q100. How does Spring Data create a repository implementation from an interface? At startup Spring Data scans for interfaces extending `Repository`, and for each it creates a **proxy** backed by `SimpleJpaRepository` (the generic CRUD implementation). For each custom method it parses the name into a query (derived query), or uses `@Query`, at startup — so a typo in a method name fails fast. Calls go through the proxy, which also applies exception translation and transactions. The inherited CRUD methods are transactional (read methods read-only), but query methods you declare yourself get no transaction settings by default. Annotate them, or the repository interface, with `@Transactional(readOnly = true)`. → *More:* §8.8 Repositories · §8.9 Derived queries

Q101. Derived query vs `@Query` (JPQL) vs native query? Derived queries come from the method name (`findByStatusAndCreatedAtAfter`) — great for simple filters, unreadable for complex ones. `@Query` holds JPQL (queries written against entities and fields, portable across databases) — use for joins, fetch joins, projections. **Native queries** (`nativeQuery = true`) are plain SQL — use for database-specific features (window functions, `SKIP LOCKED`, JSON operators) at the cost of portability. Updates need `@Modifying` plus a transaction. → *More:* §8.9 Derived queries · §8.10 @Query: JPQL, native SQL and @Modifying · Ch 18 Q84

Q102. ⋆ What is the N+1 select problem? How do you detect and fix it? Loading N parent rows with one query, then touching a lazy association on each one, fires N extra queries — 1 + N in total. Detect it with SQL logging, Hibernate statistics (`hibernate.generate_statistics`), or a test asserting the query count. Fixes: `JOIN FETCH` in a JPQL query, `@EntityGraph(attributePaths = "items")` on the repository method, batch fetching (`hibernate.default_batch_fetch_size=50` loads lazy collections in groups via `IN` lists), or a DTO projection that selects exactly the needed columns. Beware: fetch-joining a collection together with pagination makes Hibernate paginate in memory.

```
@Query("select o from Order o join fetch o.items where o.status = :status")
List<Order> findWithItems(@Param("status") Status status);
```

→ *More:* §8.7 The N+1 problem · Ch 18 Q83 · §18.17 Debugging scenarios

Q103. ⋆ Lazy vs eager loading? What are the default fetch types? Eager loads the association immediately with the owner; **lazy** loads it on first access through a proxy. Defaults: `@ManyToOne` and `@OneToOne` are EAGER, `@OneToMany` and `@ManyToMany` are LAZY. Best practice: make everything lazy (`fetch = FetchType.LAZY` on `@ManyToOne`) and fetch what each use case needs explicitly with fetch joins or entity graphs. Eager defaults cause hidden extra queries and cannot be turned off per query. → *More:* §8.6 Fetch types and LazyInitializationException · Ch 18 Q82

Q104. ⋆ What causes `LazyInitializationException`, and how do you fix it? A lazy association is accessed after the persistence context (Hibernate `Session`) that loaded the entity has closed — typically in a controller or during JSON serialization, after the `@Transactional` service method returned. Fix it by fetching what you need inside the transaction (fetch join, `@EntityGraph`) and mapping to a DTO before returning. Don't "fix" it with `FetchType.EAGER`, Open Session in View or `hibernate.enable_lazy_load_no_trans` — they hide N+1 problems. → *More:* §8.6 Fetch types and LazyInitializationException · Ch 18 Q82 · §18.17 Debugging scenarios

Q105. ⋆ What is the persistence context? What is dirty checking? The persistence context is the set of managed entity instances an `EntityManager` tracks — at most one Java object per database row (it acts as the **first-level cache**). At flush time Hibernate compares each managed entity with the snapshot taken when it was loaded and issues `UPDATE`s for changed ones: that is **dirty checking**. So inside a transaction you don't need to call `save()` after changing a loaded entity — the change is written automatically at commit. → *More:* §8.4 The persistence context and dirty checking · Ch 18 Q77

Q106. What are the JPA entity states (lifecycle)? Transient/new — created with `new`, not tracked. **Managed/persistent** — associated with a persistence context; changes are tracked. **Detached** — was managed, but the context closed (or `detach`/`clear` was called); changes are not tracked. **Removed** — scheduled for deletion at flush. `persist` makes new → managed; `merge` copies a detached object's state onto a managed instance and returns that instance. → *More:* §8.4 The persistence context and dirty checking · Ch 18 Q78

Q107. ⋆ What is the first-level cache vs the second-level cache in Hibernate? The **first-level cache** is the persistence context: per `EntityManager`/transaction, always on, guarantees one object per row and repeatable reads within the transaction. The **second-level cache** is optional and shared across sessions in the whole application (or cluster): entities marked `@Cache`/`@Cacheable` are stored by id in a provider such as Ehcache, Infinispan or Hazelcast via JCache. It helps for read-mostly reference data; it does not cache query results unless you also enable the query cache. Stale data across instances is the main risk.

Q108. `save()` vs `saveAndFlush()` vs `persist()` vs `merge()`? `save()` (Spring Data) calls `persist` if the entity is new and `merge` otherwise; the SQL is sent at flush time (usually commit). `saveAndFlush()` also flushes immediately — useful to surface constraint violations at that line, or before a native query. `persist` (JPA) attaches a new object and makes *that instance* managed; `merge` returns a *different*, managed instance, so keep using the returned object. Calling `save()` on an already managed entity is unnecessary.

Follow-up — How does `save()` decide between INSERT and UPDATE? `isNew()` decides. If the entity has a `@Version` field of a wrapper type, it is new when the version is `null`. Otherwise it is new when its `@Id` is `null` (or `0` for primitive ids). With an assigned id (e.g. a UUID you set yourself), `save()` thinks the entity exists, calls `merge`, and Hibernate runs a `SELECT` first. Fix with a `@Version` field or by implementing `Persistable<ID>`. → *More:* §8.4 The persistence context and dirty checking · §8.8 Repositories · Ch 18 Q79

Q109. `findById()` vs `getReferenceById()` (Hibernate `get()` vs `load()`)? `findById()` (JPA `find`) hits the database (or the persistence context) immediately and returns `Optional.empty()` if the row is missing. `getReferenceById()` (JPA `getReference`, Hibernate's old `load()`) returns an uninitialized **proxy** without a query; the row is read only when you touch a non-id field, and a missing row then throws `EntityNotFoundException`. Use the reference to set a foreign key (`order.setCustomer(repo.getReferenceById(id))`) without loading the customer. In Hibernate 7, `Session.load()` was removed in favour of `getReference()` (same meaning), and `Session.get()` is deprecated in favour of the JPA-standard `find()`. → *More:* §8.8 Repositories

Q110. What are the id generation strategies? Which is best for performance? `IDENTITY` (database auto-increment column): simple, but Hibernate must insert immediately to learn the id, which **disables JDBC batch inserts**. `SEQUENCE` (database sequence) with a `allocationSize` such as 50: Hibernate fetches ids in blocks, so batching works — best for PostgreSQL and Oracle. `TABLE`: portable but slow. `UUID` (`@GeneratedValue(strategy = GenerationType.UUID)` or `@UuidGenerator`): generated in the app, good for distributed systems; prefer time-ordered UUIDs (version 7) for index locality. → *More:* §8.3 Entities and ids

Q111. ⋆ Explain `@OneToMany`, `@ManyToOne`, `@ManyToMany` and the owning side (`mappedBy`). In a bidirectional relationship, the **owning side** is the one whose mapping controls the foreign key column — for one-to-many it is always the `@ManyToOne` side. The other side declares `mappedBy = "order"` and is ignored when writing; setting only the inverse side does not save the link. Keep both sides in sync with helper methods (`addItem` sets `item.setOrder(this)`). For `@ManyToMany`, prefer an explicit join entity when the link has its own data (quantity, date). → *More:* §8.5 Relationships · Ch 18 Q80

Q112. What does `cascade` do? `CascadeType.REMOVE` vs `orphanRemoval = true`? `cascade` propagates entity operations (`PERSIST`, `MERGE`, `REMOVE`, `ALL`) from parent to children, so saving an order also saves its

items. `CascadeType.REMOVE` deletes children when the **parent** is deleted. `orphanRemoval = true` also deletes a child when it is **removed from the collection**. Use cascades only for true parent–child aggregates, never on `@ManyToOne` (deleting an order item would delete the product). → *More:* §8.5 Relationships · Ch 18 Q81

Q113. What are projections in Spring Data JPA? Projections return only some columns instead of whole entities: **interface projections** (an interface with getters matching properties; Spring builds a proxy), **class/record (DTO) projections** (constructor expression in JPQL: `select new com.acme.OrderSummary(o.id, o.total) from Order o`, or a record as the return type), and **dynamic projections** (`<T> List<T> findByStatus(Status s, Class<T> type)`). They are faster and avoid lazy-loading issues because nothing is managed. → *More:* §8.12 Projections · Ch 18 Q86

Q114. `Page` vs `Slice` vs `List` return types? `Page<T>` contains the content and runs an extra `COUNT` query for the total (needed for "page 3 of 17"). `Slice<T>` only knows whether a next page exists (it fetches size + 1 rows) — cheaper, good for infinite scroll. `List<T>` with a `Pageable` just returns the rows. For very large tables, `COUNT` itself can be the slow part. → *More:* §8.11 Pagination and sorting · Ch 18 Q85

Q115. How do you write dynamic queries with optional filters (search screens)? Use **Specifications** (`JpaSpecificationExecutor` + `Specification<T>` lambdas combined with `and`/`or`, built on the Criteria API; start from `Specification.unrestricted()` — Spring Data JPA 4 (release train 2025.1) no longer accepts `null` specifications, so the old `Specification.where(null)` idiom is gone), **Query by Example** (`Example.of(probe)`), **Querydsl**, or for full control `JdbcClient` with a built SQL string and bound parameters. Never concatenate user input into JPQL/SQL — that opens SQL injection. → *More:* §8.10 @Query: JPQL, native SQL and @Modifying

Q116. ⋆ How does optimistic locking work? Optimistic vs pessimistic locking? Optimistic: add a `@Version` column; every update runs `UPDATE ... SET version = version + 1 WHERE id = ? AND version = ?`. If another transaction changed the row first, zero rows match and Spring throws `ObjectOptimisticLockingFailureException` — you retry or return 409 Conflict. No locks are held, so it suits low-contention data and web "edit forms". **Pessimistic**: `@Lock(LockModeType.PESSIMISTIC_WRITE)` issues `SELECT ... FOR UPDATE`, blocking other writers until commit — use for hot rows (inventory counters) where conflicts are frequent; keep transactions short to avoid deadlocks. → *More:* §8.14 Locking: optimistic and pessimistic · Ch 18 Q87

Q117. What is Open Session in View (OSIV), and should you disable it? OSIV keeps the persistence context (and a database connection) open for the whole web request, so lazy loading works in controllers and during JSON rendering. Boot enables it by default and logs a warning (`spring.jpa.open-in-view`). Disable it in REST services: it holds connections during slow rendering, hides N+1 queries, and makes data access happen outside transactions. Fetch what you need in the service instead. → *More:* §8.15 Open Session in View · Ch 18 Q88

Q118. Why is `spring.jpa.hibernate.ddl-auto=update` not recommended in production? How do you manage the schema? `update` only adds things, never removes or renames safely, can't migrate data, isn't reviewed or versioned, and behaves differently across environments. Use a migration tool — **Flyway** (versioned SQL files `V1__init.sql`, applied once, recorded in a history table) or **Liquibase** — and set `ddl-auto=validate` (or `none`) so Hibernate only checks that the mappings match. Boot's default is `create-drop` for an embedded database when no Flyway/Liquibase is present, and `none` otherwise. → *More:* §8.16 Schema migrations with Flyway · §9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase · Ch 18 Q89

Q119. How should a JPA entity implement `equals()` and `hashCode()`? Don't use Lombok's `@Data`/`@EqualsAndHashCode` over all fields (it touches lazy associations and changes as fields change). Use a natural business key if one exists; otherwise compare by id only when it is non-null, and return a constant `hashCode` (for example `getClass().hashCode()`) so the hash doesn't change when the id is assigned on persist. This keeps entities stable inside `HashSet`s across state changes. → *More:* §8.3 Entities and ids · Ch 18 Q90

Q120. How do you implement auditing (created/modified by and at) and soft delete? Auditing: `@EnableJpaAuditing`, `@EntityListeners(AuditingEntityListener.class)` and fields annotated `@CreatedDate`, `@LastModifiedDate`, `@CreatedBy`, `@LastModifiedBy`; an `AuditorAware` bean supplies the current user (often from the `SecurityContext`). For full history use Hibernate Envers. Soft delete: mark rows deleted

instead of deleting them — Hibernate's `@SoftDelete` (6.4+) rewrites deletes into updates and filters deleted rows from queries; the older approach is `@SQLDelete` plus `@SQLRestriction`. Remember unique constraints and indexes must account for deleted rows. → *More:* §8.13 Auditing

Q121. How do you insert or update thousands of rows efficiently with JPA? Enable JDBC batching (`spring.jpa.properties.hibernate.jdbc.batch_size=50`, plus `order_inserts`/`order_updates`), use `SEQUENCE` ids (IDENTITY disables insert batching), and in a loop `flush()` and `clear()` the `EntityManager` every batch so the persistence context doesn't grow without limit. For bulk updates use a single JPQL/SQL `UPDATE` with `@Modifying`. For really large loads, skip JPA: `JdbcClient`/`JdbcTemplate.batchUpdate`, PostgreSQL `COPY`, or Spring Batch. → *More:* §8.18 Performance checklist · §7.3 JdbcClient and JdbcTemplate

Q122. How do you stream or process a very large result set without running out of memory? Return a `Stream<T>` from a repository method (inside a read-only transaction, closed with try-with-resources) with a JDBC fetch size hint, or page through with keyset pagination (`where id > :lastId order by id limit 1000`), detaching/clearing as you go. With plain JDBC, `JdbcClient...query().stream()` or a `RowCallbackHandler` processes row by row. Avoid `findAll()` on large tables. → *More:* §8.18 Performance checklist · §7.3 JdbcClient and JdbcTemplate

Q123. How do you configure multiple data sources in Spring Boot? Define each `DataSource` yourself — bind each from its own prefix with `@ConfigurationProperties("app.datasource.orders")` and `DataSourceBuilder`, mark one `@Primary` — because Boot's auto-configuration backs off once you define one. For JPA, create a separate `LocalContainerEntityManagerFactoryBean` and `JpaTransactionManager` per source, and point each `@EnableJpaRepositories(basePackages = ..., entityManagerFactoryRef = ..., transactionManagerRef = ...)` at its packages. A transaction spans one data source only; across two you need a different design (outbox, saga), not XA, in most modern systems. For read replicas, an `AbstractRoutingDataSource` can route read-only transactions. → *More:* §7.2 DataSource and HikariCP

Q124. What is HikariCP, and how do you size the connection pool? HikariCP is Boot's default JDBC connection pool: it keeps database connections open and lends them to threads, because opening a connection per query is slow. Default `maximum-pool-size` is 10. Bigger isn't better — the database has limited cores; a starting formula is about (CPU cores × 2) + effective disk spindles, then measure. Set `connection-timeout` so callers fail fast, and remember total connections = pool size × instances. With virtual threads, the pool becomes the real concurrency limit. → *More:* §7.2 DataSource and HikariCP · Ch 18 Q64

Q125. `JdbcClient` vs `JdbcTemplate` vs JPA — when would you use plain JDBC? `JdbcTemplate` is the classic template for SQL with callbacks; `JdbcClient` (Framework 6.1+) is a fluent wrapper over it with named parameters and record mapping. Choose JDBC when you want full control over SQL, reporting queries, bulk operations, or a simple model without lazy loading surprises; choose JPA when you have a rich domain model with relationships and want dirty checking and caching. Spring Data JDBC is a middle way: repositories without lazy loading or a persistence context. → *More:* §7.3 JdbcClient and JdbcTemplate · §9.2 Spring Data JDBC and aggregates · Ch 18 Q63 · Ch 18 Q91

Transactions

Q126. ⋆ What is a transaction? Explain ACID. A transaction groups database operations into one unit that either fully happens (commit) or not at all (rollback). **Atomicity**: all or nothing. **Consistency**: constraints hold before and after. **Isolation**: concurrent transactions don't see each other's partial work (to a configurable degree). **Durability**: once committed, data survives crashes. → *More:* §7.7 How a Spring transaction works

Q127. ⋆ How does `@Transactional` work internally? Spring wraps the bean in a proxy. On a call, the `TransactionInterceptor` reads the attributes (propagation, isolation, timeout, read-only, rollback rules), asks the `PlatformTransactionManager` (e.g. `JpaTransactionManager`) to begin or join a transaction, binds the connection/`EntityManager` to the current **thread** (via `ThreadLocal`), calls your method, then commits, or rolls back if a matching exception escaped. Code on another thread does not see that transaction. → *More:* §7.7 How a Spring transaction works · Ch 18 Q65

Q128. ⋆ What are the propagation types? Explain `REQUIRED` vs `REQUIRES_NEW` vs `NESTED`. Propagation decides what happens when a transactional method is called while a transaction may already exist.

REQUIRED (default): join the existing one or start one. **REQUIRES_NEW**: suspend the outer transaction and run a completely independent one that commits or rolls back on its own (needs a second connection). **NESTED**: run inside the outer transaction with a savepoint, so the inner part can roll back alone but commits only with the outer one (JDBC savepoints — reliably supported with JDBC-based transaction managers such as `DataSourceTransactionManager`, not a general JPA feature). Others: **SUPPORTS** (join if present, else run without), **MANDATORY** (must exist or exception), **NOT_SUPPORTED** (suspend and run without), **NEVER** (exception if one exists). → *More:* §7.9 Propagation · Ch 18 Q68

Q129. When would you use `REQUIRES_NEW`? What is the risk? For work that must survive the caller's rollback — an audit log entry or a "failed attempt" record. Risks: it takes a second connection while the first is held, so under load it can exhaust the pool and deadlock the app; and the inner commit is not undone if the outer transaction later rolls back. Prefer `@TransactionalEventListener` or an outbox when possible. → *More:* §7.9 Propagation

Q130. ⋆ What are the isolation levels? What problems does each prevent? Anomalies: **dirty read** (seeing uncommitted data), **non-repeatable read** (re-reading a row gives a different value), **phantom read** (re-running a query returns new rows), plus **lost update** (two writers overwrite each other). Levels: `READ_UNCOMMITTED` (allows all), `READ_COMMITTED` (no dirty reads; PostgreSQL/Oracle default), `REPEATABLE_READ` (also no non-repeatable reads; MySQL InnoDB default; on PostgreSQL it is snapshot isolation and also prevents phantoms), `SERIALIZABLE` (behaves as if transactions ran one by one; may abort with serialization failures you must retry). Spring's `Isolation.DEFAULT` uses the database's default. → *More:* §7.11 Isolation levels · Ch 18 Q66

Q131. ⋆ A `@Transactional` method throws a checked exception. Does it roll back? No — by default Spring rolls back only on **unchecked** exceptions (`RuntimeException`) and `Error`s; a checked exception **commits** the transaction. Change it with `@Transactional(rollbackFor = Exception.class)` (or `noRollbackFor` for the reverse). Also, if you catch the exception inside the method and don't rethrow, nothing rolls back. → *More:* §7.10 Rollback rules · Ch 18 Q67 · §18.15 Proofs: five tricky answers, run

Q132. ⋆ List the common `@Transactional` pitfalls — why might it not work?

1. **Self-invocation** — calling the method via `this` bypasses the proxy. 2) The method is **private, final or static**, or the class is final (CGLIB can't override); with a JDK interface proxy, the method must be public and declared on the interface. 3) The object isn't a Spring bean (created with `new`). 4) The exception is **checked**, or it's **caught and swallowed**. 5) Work runs on **another thread** (`@Async`, `CompletableFuture`, parallel stream) — the transaction is thread-bound. 6) Wrong transaction manager with several data sources. 7) Calling it from `@PostConstruct` (proxy not ready). 8) MySQL MyISAM tables (no transactions). 9) `readOnly`/propagation settings that don't mean what you think. → *More:* §7.13 Transaction pitfalls · Ch 18 Q71 · §18.17 Debugging scenarios

Q133. What is `UnexpectedRollbackException` ("Transaction silently rolled back because it has been marked as rollback-only")? An inner `REQUIRED` method threw a runtime exception, so Spring marked the **shared** transaction rollback-only. The outer method caught the exception and tried to commit; Spring can't commit a rollback-only transaction, so it rolls back and throws `UnexpectedRollbackException`. Fix: don't catch-and-continue around a participating method — let it fail, or run the inner work in `REQUIRES_NEW` if it really is independent. → *More:* §7.9 Propagation · Ch 18 Q69

Q134. What does `@Transactional(readOnly = true)` do? It is a hint: Hibernate sets the session to read-only (no dirty-checking snapshots, flush mode MANUAL — less memory and CPU), the JDBC connection is marked read-only (some drivers/databases optimize, and routing data sources can send it to a replica). It does **not** guarantee writes are impossible on every database. Spring Data's inherited CRUD read methods (`findById`, `findAll`) are read-only by default; your own declared query methods are not. Put `@Transactional(readOnly = true)` on query service methods. → *More:* §7.12 readOnly and timeout · Ch 18 Q70

Q135. Where should `@Transactional` go — controller, service or repository? Class or method? On the **service** layer, around one business use case, so several repository calls commit together. Not on controllers (transactions would span HTTP concerns) and usually not added to repositories (they already have defaults). On a class it applies to all public methods (with class-based proxies since Framework 6.0, also protected and package-private ones); a method-level annotation overrides it. Annotate concrete classes rather than interfaces. → *More:* §7.8 @Transactional attributes · Ch 18 Q72

Q136. Can you call an external API or send a message inside a `@Transactional` method? Avoid it. A slow HTTP call holds a database connection and row locks for its whole duration, and the call cannot be

rolled back if the transaction later fails — or the message is sent but the data never commits. Do the remote call before or after the transaction, or record the intent in an **outbox** table inside the transaction and publish after commit (`@TransactionalEventListener(phase = AFTER_COMMIT)` or an outbox relay). → *More:* §7.15 @TransactionalEventListener · §14.17 The transactional outbox · Ch 18 Q74

Q137. What is `@TransactionalEventListener`? An event listener tied to the publishing transaction's outcome: by default it runs **after commit** (other phases: `BEFORE_COMMIT`, `AFTER_ROLLBACK`, `AFTER_COMPLETION`). Use it to send emails or publish messages only when the data was really saved. It runs on the same thread after commit, outside the transaction; to write to the database from it, use `REQUIRES_NEW`. If no transaction is active the event is dropped unless `fallbackExecution = true`. → *More:* §7.15 @TransactionalEventListener · §14.3 @TransactionalEventListener: events tied to a commit

Q138. Declarative vs programmatic transactions — when do you use `TransactionTemplate`? Declarative (`@Transactional`) is the default — less code, consistent. Use `TransactionTemplate` (or `PlatformTransactionManager` directly) when you need a transaction around **part** of a method, a loop that commits every N items, different settings decided at runtime, or to work around self-invocation.

```
transactionTemplate.executeWithoutResult(status -> {
    repo.save(order);
    if (!valid(order)) status.setRollbackOnly();
});
```

→ *More:* §7.14 Programmatic transactions: TransactionTemplate · Ch 18 Q73

Q139. Does `@Transactional` work with `@Async` or in a new thread? No — a transaction is bound to one thread. An `@Async` method runs on another thread, so it does not join the caller's transaction; if it is itself `@Transactional`, it starts its own. The caller may also commit before or after the async work runs, so the async method might not yet see the caller's uncommitted data. Pass ids, not entities, to async code. → *More:* §14.4 @Async and executors · §7.13 Transaction pitfalls

Q140. How do you handle deadlocks, serialization failures or optimistic-lock conflicts? They are expected under concurrency: retry the **whole transaction** (not a single statement) with a small, jittered backoff and a limit — for example with Framework 7 `@Retryable` on a method that calls the `@Transactional` method (the retry must be *outside* the transaction). Reduce them by keeping transactions short and touching rows in a consistent order. → *More:* §7.13 Transaction pitfalls · Ch 18 Q75

Q141. ⋆ How do you handle a transaction that spans multiple microservices (distributed transactions)? Avoid two-phase commit (2PC/XA): it couples services and blocks on failures. Use the **saga** pattern: a sequence of local transactions, each publishing an event; on failure, **compensating actions** undo earlier steps (refund the payment, release the stock). Choreography = services react to each other's events; orchestration = a coordinator tells each service what to do. Combine with the **transactional outbox** so each local commit and its event are atomic, and make consumers idempotent. → *More:* §16.11 Messaging between services · §14.17 The transactional outbox · Ch 18 Q158

Caching

Q142. ⋆ How do you enable caching in Spring Boot? How does the cache abstraction work? Add `@EnableCaching` and a cache provider (Caffeine for in-memory, Redis for shared; without one Boot uses a simple `ConcurrentHashMap`). Annotate methods: `@Cacheable("products")` checks the cache first and stores the result on a miss; `@CachePut` always runs the method and updates the cache; `@CacheEvict` removes entries (`allEntries = true` clears the cache); `@Caching` groups several. It's proxy-based, so the usual proxy limits apply. The `CacheManager` abstraction lets you swap providers without changing code.

Follow-up — `@Cacheable` vs `@CachePut` vs `@CacheEvict`? `@Cacheable` may skip the method (on a hit). `@CachePut` never skips it; it writes the result into the cache — use on update methods so readers see fresh data. `@CacheEvict` deletes entries — on delete methods, or when you prefer "invalidate and reload" over "write-through". Don't put `@Cacheable` and `@CachePut` on the same method. → *More:* §9.8 The caching abstraction · Ch 18 Q92

Q143. How is the cache key generated? How do you customize it? By default from the method parameters: no params → `SimpleKey.EMPTY`, one param → that value, several → a `SimpleKey` of all of them. Customize

with SpEL: `@Cacheable(cacheNames = "product", key = "#id")` or `key = "#req.sku + ':' + #req.region"`, or with a custom `KeyGenerator` bean. Key objects need correct `equals`/`hashCode` (records are ideal).

Follow-up — `condition` vs `unless` in `@Cacheable`? `condition` is evaluated **before** the call on the arguments: if false, the cache is bypassed entirely. `unless` is evaluated **after** the call and can use `#result`: if true, the result is not stored (for example `unless = "#result == null"`). → *More:* §9.8 The caching abstraction · Ch 18 Q93 · Ch 18 Q94

Q144. Why is my `@Cacheable` not working? Usual causes: `@EnableCaching` missing; self-invocation (called from the same class); the method isn't public or the bean is created with `new`; the key differs each time (an object without `equals`/`hashCode`, or a timestamp argument); the cache provider isn't on the classpath or the cache name isn't configured (Caffeine with `spring.cache.cache-names`); or values aren't serializable for Redis. → *More:* §9.8 The caching abstraction · Ch 18 Q95

Q145. Local cache (Caffeine) vs distributed cache (Redis) — when to use which? Caffeine lives in each JVM: nanosecond reads, no network, but each instance has its own copy (inconsistency across instances, memory per instance). **Redis** is shared: all instances see the same data and it survives restarts, but every read is a network call and values must be serialized. Use local for small, read-heavy, tolerant-of-staleness data; Redis for shared session/state, larger datasets or when consistency across instances matters. A two-level setup combines both. → *More:* §9.8 The caching abstraction · §9.6 Spring Data Redis (not run — needs Docker) · Ch 18 Q96

Q146. How do you set a TTL and handle cache invalidation? What problems should you plan for? Set expiry per provider: Caffeine `spring.cache.caffeine.spec=maximumSize=10000,expireAfterWrite=10m`; Redis `spring.cache.redis.time-to-live=10m` or a `RedisCacheManagerBuilderCustomizer` for per-cache TTLs. Plan for: **stale data** (evict on writes, short TTLs); **cache stampede** (many threads miss at once — use `@Cacheable(sync = true)` or request coalescing); **cache penetration** (repeated lookups of missing keys — cache null/negative results briefly); **avalanche** (many keys expiring together — add jitter to TTLs); and unbounded memory growth (always set a maximum size). → *More:* §9.8 The caching abstraction · Ch 18 Q97

Q147. Cache-aside vs write-through vs write-behind? Cache-aside (what `@Cacheable` does): read from cache, on miss load from the database and populate the cache; writes go to the database and evict the cache. **Write-through**: every write updates the cache and database together (`@CachePut`). **Write-behind**: writes go to the cache first and are flushed to the database asynchronously — fast, but data can be lost on a crash. Most Spring apps use cache-aside with eviction on writes. → *More:* §9.8 The caching abstraction

---

Spring Security: filter chain, authentication, JWT and OAuth2

Q148. What happens when you add `spring-boot-starter-security` with no configuration? Every endpoint requires authentication; Boot creates one in-memory user `user` with a random password printed at startup; form login and HTTP Basic are enabled; CSRF protection, security headers (`X-Content-Type-Options`, `X-Frame-Options`, cache control, HSTS on HTTPS) and session-fixation protection are on. Defining your own `SecurityFilterChain` bean replaces the default chain; defining a `UserDetailsService` removes the generated user. → *More:* §10.1 Vocabulary and what the starter does by default · Ch 18 Q98

Q149. ⋆ Explain the Spring Security architecture (filter chain). How does a request flow through it? The servlet container calls `DelegatingFilterProxy` (a standard filter registered by Boot), which delegates to Spring's `FilterChainProxy` bean. `FilterChainProxy` picks the first `SecurityFilterChain` whose request matcher fits the URL and runs its ordered filters: CORS, CSRF, logout, authentication filters (`UsernamePasswordAuthenticationFilter`, `BasicAuthenticationFilter`, `BearerTokenAuthenticationFilter`), session management, `ExceptionTranslationFilter` (turns security exceptions into 401/403), and finally `AuthorizationFilter` (checks access rules). Only then does the request reach the `DispatcherServlet`.

Follow-up — What is `DelegatingFilterProxy`, and why is it needed? The servlet container creates filters itself and doesn't know about Spring beans. `DelegatingFilterProxy` is a plain servlet filter that, on each request, looks up a bean (`springSecurityFilterChain`, the `FilterChainProxy`) in the application context and delegates to it — bridging the servlet world and the Spring world so security filters can be beans with injected dependencies. → *More:* §10.2 Architecture: the filter chain · Ch 18 Q99

Q150. ⋆ How do you configure Spring Security today? (`WebSecurityConfigurerAdapter` is gone) Declare one or more `SecurityFilterChain` beans using the lambda DSL. `WebSecurityConfigurerAdapter` was removed in Security 6; Security 7 also removed the `.and()` chaining and `authorizeRequests()`.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(a -> a
            .requestMatchers("/actuator/health", "/public/**").permitAll()
            .requestMatchers(HttpMethod.DELETE, "/api/**").hasRole("ADMIN")
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(Customizer.withDefaults()))
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .build();
}
```

Rules are checked in order, so put specific matchers before general ones. → *More:* §10.3 Configuring a chain: the lambda DSL in Spring Security 7 · Ch 18 Q100

Q151. ⋆ Authentication vs authorization? Authentication answers "who are you?" — verifying identity with a password, token or certificate. **Authorization** answers "what may you do?" — checking roles or permissions for a URL or method. Authentication happens first; its result (an `Authentication` with authorities) is what authorization decisions use. → *More:* §10.1 Vocabulary and what the starter does by default

Q152. How does authentication work internally? (`AuthenticationManager`, `AuthenticationProvider`, `UserDetailsService`) An authentication filter extracts credentials into an unauthenticated `Authentication` token and passes it to the `AuthenticationManager` (usually `ProviderManager`), which asks each `AuthenticationProvider` that supports that token type. `DaoAuthenticationProvider` loads the user through your `UserDetailsService.loadUserByUsername`, checks the password with the `PasswordEncoder`, and returns an authenticated token with authorities. The filter stores it in the `SecurityContext` (held in `SecurityContextHolder`, thread-local by default) and saves it to the session if sessions are used. → *More:* §10.4 Authentication: users, providers, form login and Basic · Ch 18 Q101

Q153. What is `SecurityContextHolder`? How do you get the current user? `SecurityContextHolder` holds the `SecurityContext` for the current thread, which contains the `Authentication` (principal, authorities). In a controller, inject `@AuthenticationPrincipal UserDetails user` (or `Jwt jwt` for token auth), or an `Authentication` parameter; in other code, `SecurityContextHolder.getContext().getAuthentication()`. Because it's thread-local, it doesn't follow you into `@Async` threads unless you use `DelegatingSecurityContextExecutor` or a context-propagating task decorator. → *More:* §10.10 Sessions, the security context and logout · Ch 18 Q113

Q154. How should passwords be stored? Why BCrypt? Never store plain text or fast hashes (MD5, SHA-256). Use an adaptive, salted, slow hash — **BCrypt**, **Argon2**, **scrypt** or **PBKDF2** — so brute-forcing a leaked database is expensive. Spring's `DelegatingPasswordEncoder` (from `PasswordEncoderFactories.createDelegatingPasswordEncoder()`) stores the algorithm id as a prefix (`{bcrypt}$2a$...`), so you can upgrade algorithms later and re-hash on login. → *More:* §10.5 Password storage: hashing with DelegatingPasswordEncoder · Ch 18 Q102

Q155. ⋆ Roles vs authorities? `hasRole` vs `hasAuthority`? Both are `GrantedAuthority` strings. A role is just an authority with the prefix `ROLE_`. `hasRole("ADMIN")` checks for `ROLE_ADMIN`; `hasAuthority("orders:write")` checks the exact string. Roles are coarse ("ADMIN", "USER"); authorities are fine-grained permissions or OAuth2 scopes (`SCOPE_orders.read`). → *More:* §10.6 Authorizing requests: roles, authorities, 401 vs 403 · Ch 18 Q103

Q156. ⋆ 401 vs 403? 401 Unauthorized really means *unauthenticated*: no or invalid credentials — the client should log in or send a valid token (`AuthenticationEntryPoint` produces it). **403 Forbidden** means authenticated but not allowed (`AccessDeniedHandler` produces it). A 403 on a POST that "should" work is often a missing CSRF token. → *More:* §10.6 Authorizing requests: roles, authorities, 401 vs 403 · Ch 18 Q104

Q157. ⋆ How do you secure methods? `@PreAuthorize` vs `@Secured` vs `@RolesAllowed`? Enable with `@EnableMethodSecurity`. `@PreAuthorize("hasRole('ADMIN') or #id == authentication.name")` takes a SpEL expression and can use method arguments — the most flexible; `@PostAuthorize` checks the returned object.

`@Secured("ROLE_ADMIN")` (Spring) and `@RolesAllowed("ADMIN")` (Jakarta) only list roles, no expressions; they need `securedEnabled` / `jsr250Enabled`. Method security is proxy-based. URL rules protect endpoints coarsely; method security protects business operations wherever they're called from — use both. → *More:* §10.7 Method security · Ch 18 Q105

Q158. ⋆ What is CSRF? When can you disable CSRF protection? CSRF (Cross-Site Request Forgery): a malicious site makes the victim's browser send a state-changing request to your site, and the browser automatically attaches the victim's session cookie. Spring defends with a per-session token that must accompany every POST/PUT/DELETE. You can disable it only for **stateless APIs authenticated by a header** (bearer token) that browsers don't add automatically. If you use cookies for auth (sessions, or tokens in cookies), keep it on — Security 7 has `csrf(c -> c.spa())` for single-page apps. → *More:* §10.8 CSRF protection · Ch 18 Q106

Q159. ⋆ What is a JWT? What are its parts? A JSON Web Token is a compact, signed token: `header.payload.signature`, each part Base64URL-encoded. The **header** says the algorithm (`RS256`, `HS256`) and key id; the **payload** holds **claims** — `sub` (user), `iss` (issuer), `aud` (audience), `exp` (expiry), `iat`, `scope`/roles; the **signature** proves it was issued by the key holder and wasn't changed. It is **signed, not encrypted** — anyone can read the payload, so never put secrets in it. → *More:* §11.1 From sessions to tokens: bearer tokens and JWT · Ch 18 Q110

Q160. ⋆ How do you implement JWT authentication in Spring Boot? Preferred: don't write a filter. Make the service an **OAuth2 resource server** (`spring-boot-starter-security-oauth2-resource-server` in Boot 4), set `spring.security.oauth2.resourceserver.jwt.issuer-uri` (or `jwk-set-uri`/a public key), and call `.oauth2ResourceServer(o -> o.jwt(...))`. Spring's `BearerTokenAuthenticationFilter` reads `Authorization: Bearer ...`, `JwtDecoder` checks signature, `exp`, `nbf` and issuer, and a `JwtAuthenticationConverter` maps claims to authorities. Tokens are issued by an authorization server (Keycloak, Okta, Spring Authorization Server) or, for simple cases, by your own login endpoint using `NimbusJwtEncoder`. Add audience validation for multi-service setups. → *More:* §11.3 A JWT resource server in Spring · §11.4 Minting tokens with NimbusJwtEncoder · Ch 18 Q108

Q161. Write a custom JWT filter (the way many interviewers still ask it). Extend `OncePerRequestFilter`, read and validate the header, set the `SecurityContext`, continue the chain; register it with `http.addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)`. Mention that the built-in resource server does the same with less risk.

```
class JwtFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String h = req.getHeader("Authorization");
        if (h != null && h.startsWith("Bearer ")) {
            Jwt jwt = decoder.decode(h.substring(7));   // throws JwtException if invalid/expired: catch it,
answer 401
            var auth = new JwtAuthenticationToken(jwt, authorities(jwt));
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
        chain.doFilter(req, res);
    }
}
```

→ *More:* §11.3 A JWT resource server in Spring

Q162. ⋆ Session-based vs token-based (stateless) authentication? Sessions: the server stores login state; the browser holds a session-id cookie. Easy logout and revocation, but needs sticky sessions or a shared store (Spring Session with Redis) when scaled, and CSRF protection. **Tokens (JWT)**: the client sends a self-contained signed token on each request; the server keeps no state, so any instance can verify it — good for APIs and microservices — but revocation before expiry is hard (keep access tokens short-lived, 5–15 minutes, and use refresh tokens). → *More:* §10.10 Sessions, the security context and logout · §11.1 From sessions to tokens: bearer tokens and JWT · Ch 18 Q107

Q163. How do refresh tokens work? How do you log out or revoke a JWT? A short-lived **access token** is used on every call; a long-lived **refresh token** is sent only to the authorization server to get a new access token. To revoke: keep access tokens short, revoke/rotate refresh tokens server-side (rotation: each use issues a new refresh token and invalidates the old, detecting theft), and if you need instant

revocation keep a deny-list of token ids (`jti`) in Redis, or use opaque tokens with introspection. → *More:* §11.11 Refresh tokens, lifetimes, revocation and logout

Q164. ⋆ What is OAuth 2.0? OAuth 2.0 vs OpenID Connect? Explain the authorization code flow. OAuth 2.0 is a **delegated authorization** framework: a client app gets an access token to call an API on a user's behalf without seeing the user's password. Roles: resource owner (user), client, authorization server, resource server. **OpenID Connect** adds **authentication** on top: an **ID token** (a JWT about the user) and a userinfo endpoint — "Sign in with Google" is OIDC. Authorization code flow: the client redirects the user to the authorization server → the user logs in and consents → the server redirects back with a short-lived code → the client exchanges code (plus PKCE verifier) for tokens on a back channel. **PKCE** (Proof Key for Code Exchange) stops a stolen code from being used.

Follow-up — What OAuth2 grant types exist, and which should you use? **Authorization code + PKCE** for any app acting for a user (web, mobile, SPA via a backend). **Client credentials** for service-to-service calls with no user. **Refresh token** to renew access tokens. **Device code** for TVs/CLIs. The **implicit** and **resource owner password** grants are deprecated in OAuth 2.1 best practice; Spring Security 7 removed client support for the password grant. → *More:* §11.2 OAuth 2.0 and OpenID Connect in one picture · §11.9 OAuth2 login: "Sign in with ..." · Ch 18 Q109 · §11.10 Calling other services: client credentials and token relay

Q165. How do you implement "Sign in with Google/GitHub" (OAuth2 login) in Spring Boot? Add the OAuth2 client starter, register the provider with `spring.security.oauth2.client.registration.google.client-id` / `client-secret` (Google, GitHub, Facebook and Okta have built-in provider settings), and enable `.oauth2Login(Customizer.withDefaults())`. Spring handles the redirect, code exchange, ID token validation and creates an `OidcUser` principal in the session. Map external users to your local accounts in a custom `OidcUserService`. → *More:* §11.9 OAuth2 login: "Sign in with ..."

Q166. JWT vs opaque tokens? A **JWT** is self-contained: the resource server validates it locally with the issuer's public key — fast, no network call, but hard to revoke and its claims are visible. An **opaque token** is a random string; the resource server must call the authorization server's **introspection** endpoint to learn if it's active and what it grants — instantly revocable and private, but adds a network call per request (cache the result). → *More:* §11.7 Opaque tokens and introspection · Ch 18 Q110

Q167. How does one microservice call another securely? With the **client credentials** grant: the calling service gets its own access token from the authorization server and sends it as a bearer token (Spring's `OAuth2ClientHttpRequestInterceptor` / `@ClientRegistrationId` on HTTP interfaces handles fetching and caching). To act on behalf of the original user, **relay** the incoming token (token relay at the gateway) or use token exchange. Inside a cluster, add **mTLS** (mutual TLS — both sides present certificates), often provided by a service mesh. → *More:* §11.10 Calling other services: client credentials and token relay · Ch 18 Q111

Q168. Where should a browser single-page app keep its tokens? What is the BFF pattern? Not in `localStorage` — any XSS (cross-site scripting) script can steal it. The recommended **BFF** (Backend for Frontend) pattern keeps tokens on a server-side component (for example Spring Cloud Gateway with `oauth2Login` and `TokenRelay`); the browser only gets an `HttpOnly`, `Secure`, `SameSite` session cookie, and the BFF attaches tokens when proxying API calls. Then keep CSRF protection on. → *More:* §11.12 Where should a browser keep tokens? The BFF pattern · Ch 18 Q112

Q169. How do you protect an API against brute-force login attempts and abuse (rate limiting)? Limit attempts per account and per IP (a failed-attempts counter with lockout or growing delays), add CAPTCHA or MFA after failures, and rate-limit at the edge: Spring Cloud Gateway's `RequestRateLimiter` with Redis, Bucket4j in a filter, or an API gateway/WAF. Return 429 with `Retry-After`. Log and alert on spikes. → *More:* §11.15 Rate limiting and brute-force protection

Q170. What security headers does Spring Security add, and what common web attacks does it help prevent? Defaults: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (clickjacking), `Cache-Control: no-cache, no-store`, `Strict-Transport-Security` on HTTPS, `X-XSS-Protection: 0`. You can add a Content-Security-Policy (main XSS defence) and Referrer-Policy. Spring Security also covers CSRF, session fixation (new session id after login), and timing-safe password checks; SQL injection is prevented by parameterized queries (JPA, `JdbcClient`), and XSS by output encoding in templates. → *More:* §10.11 Security headers

Q171. Can you have several security configurations in one app (e.g. API with JWT, admin UI with form login)? Yes: declare several `SecurityFilterChain` beans, each with `securityMatcher("/api/**")` etc., and

order them with `@Order` (most specific first). `FilterChainProxy` uses the first chain that matches the request. A chain without a matcher catches everything, so it must be last. → *More:* §10.3 Configuring a chain: the lambda DSL in Spring Security 7

Q172. (Security 7) What is new in Spring Security 7? Lambda-only DSL (no `.and()`), `authorizeRequests()` and `AntPathRequestMatcher`/`MvcRequestMatcher` removed (use `authorizeHttpRequests()` and `PathPatternRequestMatcher`), built-in **multi-factor authentication** support, `csrf(c -> c.spa())`, Spring Authorization Server shipped inside Security (with PKCE enabled by default), password-grant support removed, and `@ClientRegistrationId` allowed at type level on HTTP service interfaces. → *More:* §17.15 Spring Security 6 → 7 · Ch 18 Q114

Testing

Q173. How do you test a Spring Boot application? Explain the test pyramid. Many fast **unit tests** with no Spring (plain JUnit 6 + Mockito, constructor injection makes this easy); fewer **slice tests** that start only part of the context (`@WebMvcTest`, `@DataJpaTest`); a few **integration tests** with `@SpringBootTest` and real infrastructure via Testcontainers; and a handful of end-to-end/contract tests. `spring-boot-starter-test` brings JUnit, AssertJ, Mockito, Spring Test, JSONassert and Awaitility; Boot 4 adds per-technology test starters. → *More:* §12.2 The test pyramid in a Spring app · §12.3 Test dependencies in Boot 4 · Ch 18 Q115

Q174. ⋆ `@SpringBootTest` vs `@WebMvcTest` vs `@DataJpaTest`? `@SpringBootTest` loads the **whole** application context — realistic but slower; add `webEnvironment = RANDOM_PORT` to start a real server. `@WebMvcTest(OrderController.class)` loads only the web layer (controllers, advice, filters, converters, security) with `MockMvc`; services must be mocked with `@MockitoBean`. `@DataJpaTest` loads only JPA (entities, repositories, `DataSource`, Flyway), runs each test in a rolled-back transaction, and uses an embedded database unless told otherwise. → *More:* §12.6 @SpringBootTest: the whole application · §12.7 Test slices · Ch 18 Q116 · Ch 18 Q121

Q175. ⋆ `@Mock` vs `@MockBean` vs `@MockitoBean`? Why doesn't `@MockBean` compile in Boot 4? `@Mock` (Mockito) creates a mock in a plain unit test — no Spring involved. `@MockitoBean` (Spring Framework 6.2+) **replaces a bean in the Spring context** with a Mockito mock; `@MockitoSpyBean` wraps the real bean in a spy. Boot's `@MockBean`/`@SpyBean` were deprecated in Boot 3.4 and **removed in Boot 4**, so migrate to the Framework annotations. → *More:* §12.10 @MockitoBean and @MockitoSpyBean · Ch 18 Q117

Q176. How do you test a REST controller? With `@WebMvcTest` and `MockMvcTester` (AssertJ-style MockMvc, Framework 6.2+ / Boot 3.4+) or `RestTestClient` (Framework 7), mocking the service:

```
@WebMvcTest(OrderController.class)
class OrderControllerTest {
    @Autowired MockMvcTester mvc;
    @MockitoBean OrderService service;

    @Test void returns404() {
        given(service.find(1L)).willThrow(new OrderNotFoundException(1L));
        assertThat(mvc.get().uri("/api/orders/1")).hasStatus(HttpStatus.NOT_FOUND);
    }
}
```

Follow-up — `MockMvc` vs `TestRestTemplate`/`RestTestClient` vs `WebTestClient`? `MockMvc`/`MockMvcTester` call the `DispatcherServlet` in-process without a real HTTP server — fast, but filters and serialization are exercised only through the mock request. `RestTestClient` (Framework 7) and `TestRestTemplate` send real HTTP to a server on a random port — full stack. `WebTestClient` is the reactive equivalent and also works against MVC apps. With `@SpringBootTest`, these clients need explicit opt-in: `@AutoConfigureMockMvc` (as before), and in Boot 4 also `@AutoConfigureRestTestClient` or `@AutoConfigureTestRestTemplate`. A `RANDOM_PORT` test no longer gets a `TestRestTemplate` automatically. → *More:* §12.8 MockMvcTester: AssertJ-style MockMvc · §12.9 RestTestClient (Framework 7.0) · Ch 18 Q118 · §12.13 Full-stack tests on a real port

Q177. What is test context caching? Why do my integration tests get slower as I add more? The Spring TestContext Framework caches application contexts and reuses one for every test class with the **same configuration**. Each different combination of `@MockitoBean`s, profiles, properties, `@Import`s or

`@DirtiesContext` creates a new context (seconds each). Keep test configuration uniform (shared base class or meta-annotation), avoid `@DirtiesContext`, and watch the log for repeated startups. → *More:* §12.11 Test context caching · Ch 18 Q119

Q178. Does `@Transactional` on a test method roll back? Yes — in Spring tests, a test-managed transaction rolls back after each test by default (`@Commit` changes that). Caveat: with `@SpringBootTest(webEnvironment = RANDOM_PORT)` the HTTP request runs on a server thread in a **different** transaction, so the test's rollback doesn't undo it. Also, rollback means the flush may never happen — call `flush()` to catch constraint errors. → *More:* §12.6 @SpringBootTest: the whole application · Ch 18 Q120

Q179. ⋆ How do you test against a real database, Kafka or Redis? (Testcontainers) Use **Testcontainers**: it starts real services in Docker for the test. Annotate the container field with `@ServiceConnection` and Boot wires the connection properties automatically (no `@DynamicPropertySource` needed); share containers across classes so they start once. This catches SQL dialect, migration and driver issues an in-memory H2 database hides.

```
@Testcontainers @SpringBootTest
class RepoIT {
    @Container @ServiceConnection
    static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:17-alpine");
}
```

Follow-up — What is `@DynamicPropertySource`, and when do you use `@TestPropertySource` or `@ActiveProfiles`? `@DynamicPropertySource` registers properties whose values are only known at runtime (a container's random port) before the context starts. `@TestPropertySource` and `@SpringBootTest(properties = ...)` add fixed test properties with high precedence. `@ActiveProfiles("test")` activates `application-test.properties`. Each different combination is a different cached context. → *More:* §12.14 Testcontainers and @ServiceConnection · Ch 18 Q122 · §12.12 @DynamicPropertySource: properties known only at runtime · §12.15 Profiles and properties in tests

Q180. How do you test secured endpoints? With `spring-security-test`: `@WithMockUser(roles = "ADMIN")` on a test, or request post-processors such as `.with(jwt().authorities(...))`, `.with(user("ann"))` and `.with(csrf())` on MockMvc requests. Test both the allowed and the forbidden path (401 and 403). → *More:* §10.12 Testing security with spring-security-test · §11.6 Testing token security · Ch 18 Q123

Q181. How do you test a service that calls an external HTTP API? Unit level: mock the client interface. Client level: `@RestClientTest` with `MockRestServiceServer` to assert the exact request and return canned responses. Integration level: **WireMock** (a fake HTTP server) to simulate slow or failing responses and test timeouts and retries. Contract tests (Spring Cloud Contract, Pact) ensure your fake matches the real provider. → *More:* §6.8 Testing clients · Ch 18 Q61

Q182. How do you test `@Async` code, scheduled jobs or Kafka listeners? Don't sleep — use **Awaitility** (`await().atMost(5, SECONDS).untilAsserted(...)`) to poll for the expected effect. For scheduled jobs, test the job's method directly and keep the schedule as configuration. For Kafka, use `@EmbeddedKafka` or a Kafka Testcontainer, send a message and await the side effect. → *More:* §14.13 Testing with @EmbeddedKafka · §12.16 Common mistakes — checklist

Q183. How do you test custom auto-configuration or conditional beans? `ApplicationContextRunner` starts a tiny context in milliseconds per test: `.withUserConfiguration(...)`, `.withPropertyValues("feature.enabled=true")`, `.withClassLoader(new FilteredClassLoader(SomeLib.class))`, then assert `context.hasSingleBean(...)`. → *More:* §3.13 Writing your own auto-configuration and starter · Ch 18 Q124

WebFlux, async, scheduling and messaging

Q184. ⋆ What is Spring WebFlux? Spring MVC vs WebFlux — when would you choose each? WebFlux is Spring's **non-blocking, reactive** web stack, built on Project Reactor and usually running on Netty. Instead of one thread per request, a few event-loop threads handle many connections, and code returns `Mono`/`Flux` that complete when data arrives. Choose WebFlux for streaming (SSE, WebSockets), gateways and apps that fan out to many slow remote calls with an end-to-end reactive stack (R2DBC, reactive drivers). Choose MVC for most CRUD services, especially with JPA/JDBC (blocking) — and since Java 21, **MVC on virtual threads**

gives similar scalability for blocking I/O with simpler code. → *More:* §13.1 Why reactive? The cost of waiting · §13.15 Virtual threads vs WebFlux: a measured comparison · Ch 18 Q131

Q185. What is reactive programming? What is backpressure? Reactive programming composes asynchronous streams of data with operators, reacting to items as they arrive instead of blocking a thread while waiting. **Reactive Streams** is the standard contract (`Publisher`, `Subscriber`, `Subscription`, `Processor`) adopted into Java as `java.util.concurrent.Flow`. **Backpressure** means the subscriber tells the publisher how many items it can handle (`request(n)`), so a fast producer can't overwhelm a slow consumer. → *More:* §13.2 Reactive Streams: the contract · §13.7 Backpressure in practice · Ch 18 Q126

Q186. ⋆ `Mono` vs `Flux`? Both are Reactor `Publisher`s. `Mono<T>` emits 0 or 1 item then completes (or errors) — like an async `Optional`, for a single result. `Flux<T>` emits 0..N items — for lists and streams. **Nothing happens until someone subscribes**: in WebFlux the framework subscribes to what your controller returns.

Follow-up — Hot vs cold publishers? A **cold** publisher starts producing from scratch for each subscriber (a `WebClient` call runs once per subscription). A **hot** publisher emits regardless of subscribers, and late subscribers miss earlier items (a stream of price ticks, `Sinks.many().multicast()`). `share()`, `cache()` and `replay()` turn cold into hot.

Follow-up — How do you test reactive code? With `StepVerifier`: `StepVerifier.create(flux).expectNext(1, 2).expectComplete().verify()`; `withVirtualTime` for delays without waiting. Test WebFlux endpoints with `WebTestClient`. → *More:* §13.3 Project Reactor: Mono and Flux · Ch 18 Q127 · §13.9 Hot vs cold, sinks and context · §13.5 Testing with StepVerifier and virtual time · Ch 18 Q132

Q187. `map` vs `flatMap` (vs `concatMap`) in Reactor? `map` transforms each item **synchronously** (`T → R`). `flatMap` transforms each item into another publisher (`T → Mono<R>`), subscribes to them **concurrently** and merges results — order not preserved. `concatMap` does the same but one at a time, preserving order. Use `flatMap` for async calls such as a `WebClient` request per item; limit concurrency with `flatMap(fn, 8)`. → *More:* §13.4 Operators · Ch 18 Q128

Q188. What happens if you call blocking code (JDBC, `Thread.sleep`, `.block()`) in a WebFlux handler? You block one of the few event-loop threads; all other requests assigned to it stall and throughput collapses. Wrap unavoidable blocking calls with `Mono.fromCallable(...).subscribeOn(Schedulers.boundedElastic())`, use reactive drivers (R2DBC, reactive Mongo/Redis), and detect mistakes in tests with BlockHound. → *More:* §13.8 Schedulers and threads · Ch 18 Q129

Q189. `subscribeOn` vs `publishOn`? `subscribeOn` decides which scheduler the **subscription and source** run on — its position in the chain doesn't matter (first one wins). `publishOn` switches the thread for the operators **after** it in the chain. Use `subscribeOn(boundedElastic())` to move a blocking source off the event loop; use `publishOn` to hop threads mid-pipeline. → *More:* §13.8 Schedulers and threads · Ch 18 Q130

Q190. How do you handle errors in Reactor? Errors are terminal signals that flow downstream. Operators: `onErrorReturn(fallback)`, `onErrorResume(e -> otherPublisher)`, `onErrorMap(e -> new MyException(e))`, `doOnError(log)`, `retryWhen(Retry.backoff(3, Duration.ofMillis(200)))`, and `timeout(Duration)`. `switchIfEmpty` handles "no value" (e.g. return 404). In WebFlux, `@ExceptionHandler` / `@ControllerAdvice` work as in MVC. → *More:* §13.6 Error handling

Q191. ⋆ What are virtual threads, and how do they change Spring Boot apps? Virtual threads (Java 21+) are lightweight threads managed by the JVM: blocking I/O unmounts them from the carrier OS thread, so you can have hundreds of thousands. With `spring.threads.virtual.enabled=true`, Boot runs Tomcat request handling, `@Async`, `@Scheduled` and some listeners on virtual threads. Benefit: thread-per-request code scales for I/O-bound work without reactive code. Watch-outs: the **connection pool** becomes the limit, `ThreadLocal`-heavy code multiplies memory, CPU-bound work doesn't benefit, and before Java 24 `synchronized` blocks pinned the carrier thread (fixed by JEP 491 in Java 24). → *More:* §15.10 Virtual threads · §13.15 Virtual threads vs WebFlux: a measured comparison · Ch 18 Q148

Q192. ⋆ How does `@Async` work? What are its pitfalls? With `@EnableAsync`, calls to an `@Async` method go through a proxy that submits the work to a `TaskExecutor` and returns immediately (`void` or `CompletableFuture<T>`). Pitfalls: self-invocation runs it synchronously; exceptions from `void` methods are lost unless you configure an `AsyncUncaughtExceptionHandler`; the security context and transaction don't propagate to the new thread; and Boot's default executor queue is unbounded — configure

`spring.task.execution.pool.*` or use virtual threads. → *More:* §14.4 @Async and executors · §14.6 Exceptions in async methods · Ch 18 Q133

Q193. How do you call three services in parallel and combine the results? In MVC: `CompletableFuture.supplyAsync(...)` per call on a bounded executor (or virtual threads), then `CompletableFuture.allOf(a, b, c).join()` and read each result; always add timeouts (`orTimeout`). Java 25's structured concurrency (`StructuredTaskScope`) is still a preview API. In WebFlux: `Mono.zip(callA(), callB(), callC())`.

```
var price = CompletableFuture.supplyAsync(() -> pricing.get(id), executor);
var stock = CompletableFuture.supplyAsync(() -> inventory.get(id), executor);
return price.thenCombine(stock, ProductView::new).orTimeout(2, TimeUnit.SECONDS).join();
```

→ *More:* §14.4 @Async and executors · §13.4 Operators

Q194. ⋆ How do you schedule tasks? `fixedRate` vs `fixedDelay` vs `cron`? `@EnableScheduling` plus `@Scheduled` on a bean method. `fixedRate = 5000` starts every 5 s measured from the **start** of the previous run; `fixedDelay = 5000` waits 5 s after the previous run **finishes**; `cron = "0 0 2 * * *"` uses a 6-field cron expression (second first) with an optional `zone`. Durations can be strings (`fixedRateString = "PT5M"`). By default all scheduled tasks share **one thread**, so one slow job delays others — set `spring.task.scheduling.pool.size` or use virtual threads. → *More:* §14.7 Scheduling with @Scheduled · Ch 18 Q134

Q195. ⋆ A `@Scheduled` job runs on every instance of a scaled-out service. How do you run it only once? Use a distributed lock: **ShedLock** (`@SchedulerLock` with a lock table in the database or Redis), a database row lock (`SELECT ... FOR UPDATE SKIP LOCKED` to claim work), or Spring Integration's leader election. Alternatively, move the job out of the service into a Kubernetes CronJob or a workflow scheduler. Make the job idempotent anyway. → *More:* §14.7 Scheduling with @Scheduled · Ch 18 Q135

Q196. ⋆ How do you produce and consume Kafka messages with Spring Boot? Add `spring-kafka` (Boot 4: `spring-boot-starter-kafka`), configure `spring.kafka.bootstrap-servers` and (de)serializers. Produce with `KafkaTemplate.send(topic, key, value)` — the **key** picks the partition, so events for one order stay in order. Consume with `@KafkaListener(topics = "orders", groupId = "billing")`; consumers in the same group split the partitions, so concurrency is capped by the partition count. → *More:* §14.10 Setting up Spring Kafka (with JSON) · §14.11 Producing: KafkaTemplate, keys and partitions · §14.12 Consuming: @KafkaListener, concurrency and consumer groups · Ch 18 Q136

Q197. ⋆ What delivery guarantees does Kafka give? How do you handle duplicate messages (idempotent consumer)? Spring Kafka listeners give **at-least-once** delivery by default: the offset is committed after your listener returns, so a crash before the commit causes redelivery. Therefore consumers must be **idempotent**: store processed message ids (or a business key) in a table with a unique constraint inside the same transaction as the side effect, or use naturally idempotent operations (upserts). Exactly-once is achievable only for Kafka-to-Kafka flows with transactions. → *More:* §14.12 Consuming: @KafkaListener, concurrency and consumer groups · §14.16 Producer guarantees: idempotence and transactions · Ch 18 Q137

Q198. How do you handle a message that keeps failing (poison message)? Explain retries and dead-letter topics. Configure a `DefaultErrorHandler` with a `BackOff` (for example 3 retries, exponential) and a `DeadLetterPublishingRecoverer` that publishes the failed record to `orders.DLT` with exception headers, then moves on. Mark non-retryable exceptions (validation, deserialization) so they go straight to the DLT. `@RetryableTopic` gives **non-blocking** retries through delay topics so one bad message doesn't block its partition. Monitor and replay the DLT. → *More:* §14.14 Error handling: DefaultErrorHandler and the dead-letter topic · §14.15 Non-blocking retries with @RetryableTopic · Ch 18 Q138

Q199. ⋆ What is the transactional outbox pattern? (dual-write problem) Writing to the database and publishing to Kafka are two systems; one can succeed while the other fails (the **dual-write problem**). With an outbox, the service writes the business change **and** an event row into an `outbox` table in the same database transaction; a separate relay (a scheduled poller, or CDC — change data capture — with Debezium) publishes outbox rows to Kafka and marks them sent. Delivery becomes at-least-once, so consumers must be idempotent. → *More:* §14.17 The transactional outbox · Ch 18 Q139

Q200. Kafka vs RabbitMQ — when do you use which? Kafka is a distributed, partitioned, replicated **log**: messages are retained and can be replayed, ordering is per partition, very high throughput — for event

streaming, event sourcing, analytics pipelines. **RabbitMQ** is a **message broker** with flexible routing (exchanges, bindings), per-message acknowledgement, priorities and TTLs; messages are removed once consumed — for task queues and request/reply. Spring supports both (`spring-kafka`, Spring AMQP). → *More:* §14.8 Messaging concepts you need first · §14.18 RabbitMQ with Spring AMQP (configuration, not run) · Ch 18 Q140

Q201. How do you guarantee message ordering in Kafka? Kafka orders only within a partition. Use the entity id as the message key so all events for it go to the same partition; enable the idempotent producer (default in modern clients) so retries don't reorder; on the consumer side, process a partition sequentially (one consumer thread per partition) and be careful with retry topics, which break ordering. → *More:* §14.11 Producing: KafkaTemplate, keys and partitions · §14.16 Producer guarantees: idempotence and transactions

---

Actuator, observability, production and performance

Q202. ⋆ What is Spring Boot Actuator? Which endpoints do you know, and how do you secure them? Actuator adds production endpoints: `health`, `info`, `metrics`, `prometheus`, `loggers` (change levels at runtime), `env`, `configprops`, `beans`, `conditions`, `mappings`, `threaddump`, `heapdump`, `scheduledtasks`, `httpexchanges`, `shutdown` (off by default). Over HTTP only `health` is exposed by default. Expose deliberately (`management.endpoints.web.exposure.include=health,info,prometheus`), consider a separate `management.server.port` not reachable from outside, and protect everything except health with Spring Security — `env` and `heapdump` can leak secrets. → *More:* §15.2 Spring Boot Actuator · Ch 18 Q141

Q203. How do you write a custom health indicator? Implement `HealthIndicator` (or `ReactiveHealthIndicator`) as a bean and return `Health.up().withDetail(...)` or `Health.down(ex)`; its name (bean name without `HealthIndicator`) appears under `/actuator/health`. Group indicators into `liveness` and `readiness` groups. Don't make liveness depend on external systems.

```
@Component
class PaymentGatewayHealthIndicator implements HealthIndicator {
    public Health health() {
        return client.ping() ? Health.up().build() : Health.down().withDetail("gateway", "unreachable").build();
    }
}
```

→ *More:* §15.3 Health, health groups and Kubernetes probes

Q204. Liveness vs readiness probes? Liveness = "is the process healthy, or should it be restarted?" — fail only for unrecoverable states (deadlock, corrupted state). **Readiness** = "can it take traffic right now?" — false during startup, warm-up, shutdown or when a critical dependency is down; Kubernetes stops routing traffic instead of restarting. Boot exposes `/actuator/health/liveness` and `/actuator/health/readiness` (enabled by default in Boot 4; in Boot 3 only when it detects Kubernetes or with `management.endpoint.health.probes.enabled=true`). Never put the database in liveness — a database blip would restart every pod. → *More:* §15.3 Health, health groups and Kubernetes probes · Ch 18 Q142

Q205. ⋆ How do you monitor a Spring Boot app in production? What is Micrometer? The three pillars: **metrics** (Micrometer → Prometheus → Grafana dashboards and alerts), **logs** (structured JSON → ELK/Loki), **traces** (Micrometer Tracing/OpenTelemetry → Zipkin/Tempo/Jaeger). **Micrometer** is a vendor-neutral metrics facade ("SLF4J for metrics"): you record `Counter`, `Gauge`, `Timer`, `DistributionSummary` and `LongTaskTimer` meters and it exports to Prometheus, Datadog, OTLP etc. Boot auto-instruments HTTP requests, JVM, connection pools, caches and Kafka. Track the RED metrics (rate, errors, duration) per endpoint. → *More:* §15.1 Observability in one page · §15.5 Metrics with Micrometer · Ch 18 Q143

Q206. ⋆ How does distributed tracing work in Spring Boot? What replaced Spring Cloud Sleuth? Each request gets a **trace id**; each hop (HTTP call, message) is a **span** with its own id and a parent id. Ids travel in headers (W3C `traceparent`), so a tracing backend can reconstruct the whole call tree with timings. Since Boot 3, **Micrometer Tracing** (with an OpenTelemetry or Brave bridge) replaced Sleuth; auto-configured `RestClient`, `WebClient`, `KafkaTemplate` propagate context, and the trace id appears in logs for correlation. Sample a percentage in production (`management.tracing.sampling.probability`).

Follow-up — How do you trace a request and correlate logs across microservices? Use Micrometer Tracing (or OpenTelemetry) so every service propagates `traceparent` headers over HTTP and Kafka; Boot puts `traceId` and `spanId` into the logging MDC, so with structured JSON logs you can search all services' logs by one trace id and see the call tree in Zipkin/Tempo. → *More:* §15.7 Distributed tracing · §16.9 Distributed tracing across services · Ch 18 Q145 · §15.8 Structured logging

Q207. What is the Observation API (`@Observed`)? Micrometer's Observation API instruments code **once** and produces both a metric (timer) and a trace span (and log correlation) from it. Spring's own instrumentation uses it. Annotate a method with `@Observed(name = "order.place")` or wrap code in `Observation.createNotStarted(...).observe(...)`; add low-cardinality tags for metrics and high-cardinality ones only for traces. → *More:* §15.6 The Observation API and @Observed · Ch 18 Q144

Q208. What is graceful shutdown? On SIGTERM, Boot stops accepting new requests, lets in-flight requests finish (up to `spring.lifecycle.timeout-per-shutdown-phase`, default 30 s), then closes beans. Graceful shutdown is enabled by default since Boot 3.4. Readiness turns to refusing traffic first. In Kubernetes, align `terminationGracePeriodSeconds` and add a short `preStop` sleep so the load balancer stops routing before the app stops. → *More:* §15.9 Graceful shutdown · Ch 18 Q147

Q209. How do you improve Spring Boot startup time and memory usage? Measure first: `ApplicationStartup` (`BufferingApplicationStartup` + `/actuator/startup`) shows slow beans. Then: remove unused starters (Boot 4's modules help), avoid heavy work in `@PostConstruct`, lazy-initialize rarely-used beans, use **CDS / the AOT cache** (Java 24+ JEP 483: train once, start faster — Boot supports extracted jars for it), Spring AOT processing, or a **GraalVM native image** for instant startup and low memory at the cost of long builds and reflection limits. Tune the JVM for containers (`-XX:MaxRAMPercentage`). → *More:* §15.11 Faster startup: extracted jars, CDS, AOT cache, native images · Ch 18 Q149 · Ch 18 Q150

Q210. What are AOT processing and GraalVM native images? What are the trade-offs? AOT (ahead-of-time) processing runs at build time: it evaluates the configuration and generates Java source for bean definitions plus hints for reflection, proxies and resources. A **native image** compiles the app and JVM parts into a platform-specific executable: starts in tens of milliseconds with a small memory footprint, ideal for scale-to-zero and CLIs. Trade-offs: slow builds, a "closed world" (classpath fixed at build time, `@Profile`/`@ConditionalOnProperty` decided at build), libraries need reflection hints, and peak throughput may be lower than a warmed-up JIT. → *More:* §15.11 Faster startup: extracted jars, CDS, AOT cache, native images · Ch 18 Q149

Q211. How do you package a Spring Boot app as a Docker image? Either `mvn spring-boot:build-image` (Cloud Native Buildpacks — no Dockerfile, a layered, secure image) or a multi-stage Dockerfile that uses Boot's **layered jar** (`java -Djarmode=tools -jar app.jar extract --layers`) so dependencies sit in a separate cached layer from your code. Run as a non-root user, set memory flags, and use a JRE base image. → *More:* §15.11 Faster startup: extracted jars, CDS, AOT cache, native images · Ch 18 Q151

Q212. ⋆ How would you find out why an endpoint is slow?

1. Metrics: is it slow for all requests or the p99 only, and since when (deploy? traffic?) — `http.server.requests` timers. 2) Traces: which span takes the time — database, a downstream call, or the app itself. 3) Database: SQL logs/Hibernate statistics for N+1 queries, `EXPLAIN ANALYZE` for missing indexes, connection-pool wait time (`hikaricp.connections.pending`). 4) JVM: thread dumps (blocked threads, lock contention), GC pauses, CPU profiling with async-profiler or JFR (Java Flight Recorder). 5) Fix the cause (index, fetch join, cache, timeout), and re-measure. → *More:* §15.5 Metrics with Micrometer · §8.18 Performance checklist

Q213. How do you diagnose high memory usage or an `OutOfMemoryError` in a Spring Boot app? Run with `-XX:+HeapDumpOnOutOfMemoryError`, or capture `/actuator/heapdump`, and analyze in Eclipse MAT or VisualVM for the dominating objects. Usual Spring culprits: loading a huge result set (`findAll()`), an unbounded cache or `ConcurrentHashMap` used as a cache, an unbounded `@Async` queue, a persistence context growing in a batch loop (missing `clear()`), large request bodies read into memory, and `ThreadLocal`s not cleaned up. Check also container limits vs `-Xmx` (non-heap memory: metaspace, thread stacks, direct buffers). → *More:* §15.2 Spring Boot Actuator · §8.18 Performance checklist

Q214. What is on your production-readiness checklist for a Spring Boot service? Externalized config and secrets; health groups and probes; metrics, structured logs and tracing with correlation ids; graceful shutdown; timeouts and circuit breakers on every remote call; connection pool sized and monitored; Flyway migrations with `ddl-auto=validate`; OSIV off; Actuator endpoints restricted; security headers and

TLS; container image with memory limits; resource limits and autoscaling; dashboards and alerts on error rate and latency. → *More:* §15.12 Production configuration checklist · Ch 18 Q152

Microservices and Spring Cloud

Q215. ⋆ What are microservices? Monolith vs microservices — pros and cons? Microservices split a system into small services, each owning one business capability and its own data, deployed independently and communicating over the network (HTTP, messaging). Pros: independent deployment and scaling, team autonomy, fault isolation, technology freedom. Cons: distributed-systems complexity (network failures, latency, eventual consistency), harder testing and debugging, operational overhead (CI/CD, observability, platform). A **modular monolith** (Spring Modulith) is often the better start. → *More:* §16.1 The map: microservice problems and Spring answers

Q216. ⋆ How does Spring Boot/Spring Cloud support microservices? Name the main Spring Cloud components. Boot makes each service a self-contained jar with health, metrics and config. Spring Cloud adds the distributed-system patterns: **Config** (centralized configuration), **Netflix Eureka**/Consul/Kubernetes (service discovery), **LoadBalancer** (client-side load balancing), **Gateway** (API gateway), **Circuit Breaker** with Resilience4j, **OpenFeign** (declarative clients, now in maintenance), **Stream** (messaging binders), **Contract** (consumer-driven contract tests), and **Kubernetes** integration. Boot 4.0 and 4.1 use Spring Cloud 2025.1.x ("Oakwood"); Boot 4.1 needs 2025.1.2 or later. → *More:* §16.1 The map: microservice problems and Spring answers · §16.2 Release trains and versions · Ch 18 Q153

Q217. What is service discovery? How does Eureka work? Instances come and go with changing IPs, so clients look services up by **name** in a registry instead of hard-coding addresses. With **Eureka**, each service registers itself on startup and sends heartbeats; clients fetch and cache the registry and pick an instance via Spring Cloud LoadBalancer (`http://inventory-service/...` with a `@LoadBalanced RestClient.Builder`). On Kubernetes you usually don't need Eureka — Kubernetes Services and DNS do discovery. → *More:* §16.4 Service discovery: Eureka, Consul, Kubernetes · §16.5 Client-side load balancing with Spring Cloud LoadBalancer · Ch 18 Q155

Q218. ⋆ What is an API gateway? How does Spring Cloud Gateway work? An API gateway is the single entry point for clients: routing to services, authentication/token relay, rate limiting, CORS, retries, request/response rewriting and aggregation. Spring Cloud Gateway defines **routes** (id, destination URI, **predicates** such as `Path=/orders/**`, and **filters** such as `StripPrefix`, `RequestRateLimiter`, `CircuitBreaker`). It comes in a reactive (WebFlux/Netty) variant and a Server Web MVC variant. Zuul 1 is retired. → *More:* §16.7 API gateway: Spring Cloud Gateway · Ch 18 Q156

Q219. ⋆ What is the circuit breaker pattern? How do you implement it in Spring Boot? A circuit breaker wraps calls to a dependency and tracks failures. **Closed**: calls pass. When the failure rate (or slow-call rate) passes a threshold, it goes **open**: calls fail fast (or use a fallback) without hitting the struggling service. After a wait it goes **half-open** and lets a few trial calls through; success closes it again. Implement with **Resilience4j** (`@CircuitBreaker(name = "inventory", fallbackMethod = "cached")` plus `resilience4j.circuitbreaker.instances.*` properties) or Spring Cloud Circuit Breaker. Hystrix is dead. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · Ch 18 Q157

Q220. What other resilience patterns do you know? (retry, timeout, bulkhead, rate limiter, fallback) **Timeout**: never wait forever. **Retry** with exponential backoff and jitter, only for idempotent, transient failures. **Bulkhead**: cap concurrent calls per dependency so one slow service can't use every thread. **Rate limiter**: cap calls per time window. **Fallback**: a cached or default response. Order matters: Resilience4j applies retry outside the circuit breaker so the breaker counts each attempt. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · §4.7 Framework 7 core resilience: @Retryable and @ConcurrencyLimit

Q221. ⋆ How do microservices communicate? Synchronous vs asynchronous? Synchronous (REST over HTTP, gRPC): simple request/response, immediate answer, but temporal coupling — if the callee is down or slow, the caller suffers (needs timeouts, retries, circuit breakers). **Asynchronous** (Kafka, RabbitMQ events): the sender doesn't wait; services are decoupled in time, absorb load spikes, but you get eventual consistency, duplicate messages and harder debugging. Use sync for queries that need an answer now; async for propagating state changes between services. → *More:* §16.11 Messaging between services · §14.8 Messaging concepts you need first

Q222. What does Spring Cloud Config do? How do you refresh configuration without restarting? A Config Server serves properties from Git (or Vault, a database) to all services, which import it with `spring.config.import=configserver:http://config:8888`. To apply changes at runtime, beans annotated `@RefreshScope` (and `@ConfigurationProperties` beans) are rebuilt when `/actuator/refresh` is called; Spring Cloud Bus broadcasts the refresh to all instances. On Kubernetes, ConfigMaps plus a rolling restart is the simpler option. → *More:* §16.3 Centralized configuration: Spring Cloud Config · Ch 18 Q154

Q223. ⋆ What is the saga pattern? Choreography vs orchestration? A saga implements a business transaction across services as a sequence of local transactions, each with a **compensating action** that semantically undoes it. **Choreography**: each service listens for events and emits the next one (no central brain; simple for few steps, hard to follow for many). **Orchestration**: a coordinator (an orchestrator service, or a workflow engine such as Temporal or Camunda) sends commands and tracks state — easier to reason about and monitor. Both need idempotent steps and the outbox pattern. → *More:* §16.11 Messaging between services · Ch 18 Q158

Q224. What is database per service? How do you query data owned by several services? Each service owns its tables and nobody else reads them directly — this keeps services independently deployable. To combine data: API composition (a gateway/BFF calls several services and joins), or **CQRS** read models (Command Query Responsibility Segregation — a service keeps a denormalized copy of other services' data, updated by their events). → *More:* §16.1 The map: microservice problems and Spring answers · §16.11 Messaging between services

Q225. ⋆ How do you make a POST endpoint idempotent (prevent double payments on retries)? The client sends an `Idempotency-Key` header (a UUID per logical operation). The server stores the key with the request hash and the response in a table with a unique constraint (or Redis with TTL), in the same transaction as the operation. A repeated key returns the stored response instead of executing again; a concurrent duplicate hits the unique constraint and gets 409 or waits. Combine with natural unique constraints (one payment per order id). → *More:* §5.1 REST in one page · §14.12 Consuming: @KafkaListener, concurrency and consumer groups

Q226. How do you implement rate limiting across several instances of a service? Use a shared counter store: Spring Cloud Gateway's `RequestRateLimiter` (token bucket in Redis, keyed by user or API key via a `KeyResolver`), Bucket4j with a Redis/Hazelcast backend in a servlet filter, or rate limiting in the ingress/API management layer. Local, in-memory limiters (Resilience4j `RateLimiter`) only protect a single instance — fine for protecting a downstream, not for per-client quotas. → *More:* §16.7 API gateway: Spring Cloud Gateway · §11.15 Rate limiting and brute-force protection

Q227. How do you test contracts between services? Consumer-driven contract tests: the consumer writes the expected request/response as a contract; the provider's build verifies its API satisfies every contract, and consumers test against a generated stub (Spring Cloud Contract with WireMock stubs, or Pact). This catches breaking API changes before deployment, without spinning up the whole system. → *More:* §16.10 Contract testing · Ch 18 Q159

Q228. How would you deploy Spring Boot microservices to Kubernetes? Build a container image per service (buildpacks or layered Dockerfile); deploy with a Deployment (replicas, resource requests/limits, liveness/readiness probes on Actuator health groups), a Service for discovery, ConfigMaps/Secrets for configuration, an Ingress or gateway for external traffic, a HorizontalPodAutoscaler on CPU or custom metrics; use graceful shutdown and rolling updates. Spring Cloud Kubernetes can read ConfigMaps as property sources if needed. → *More:* §15.3 Health, health groups and Kubernetes probes · §16.4 Service discovery: Eureka, Consul, Kubernetes

---

Scenario, troubleshooting and coding questions

Q229. ⋆ Your application fails to start with "No qualifying bean of type ..." or "required a bean ... that could not be found". What do you check? Read the failure-analysis report: it names the injection point and the missing type. Check in order: is the class annotated (`@Service`, `@Component`) or declared with `@Bean`? Is it inside the main class's package tree (component scanning)? Is it excluded by a `@Profile` or a failed `@Conditional` (run with `--debug` and read the condition report)? Is the starter or library that provides it on the classpath? Are you injecting a concrete class while the bean is a JDK

proxy of an interface? For `NoUniqueBeanDefinitionException`, add `@Qualifier` or `@Primary`. → *More:* §18.17 Debugging scenarios · §2.5 How Spring picks a bean

Q230. The app fails with "Failed to configure a DataSource: 'url' attribute is not specified" or "Port 8080 was already in use". What is happening? The first means a JDBC/JPA starter is on the classpath, so Boot tries to auto-configure a `DataSource`, but there is no `spring.datasource.url` and no embedded database (H2) — add the URL for the active profile, add H2 for local runs, or exclude `DataSourceAutoConfiguration` if the app really needs no database. The second means another process (often a previous run) holds the port: stop it, or change `server.port` (use `0` in tests). → *More:* §7.2 DataSource and HikariCP · §3.12 Overriding and excluding auto-configuration

Q231. ⋆ Two users edit the same record at the same time and one user's changes silently disappear. How do you prevent it? That is a **lost update**. Add a `@Version` column (optimistic locking): the second save fails with `ObjectOptimisticLockingFailureException`, which you map to **409 Conflict** so the user reloads and retries. Expose the version to the client as an `ETag` and require `If-Match` on PUT/PATCH so stale edits are rejected with 412 Precondition Failed. For frequent conflicts on the same row, use a pessimistic lock or a single atomic `UPDATE`. → *More:* §8.14 Locking: optimistic and pessimistic

Q232. ⋆ Under load, your shop sells more items than are in stock. How do you fix the race condition? Two transactions read `stock = 1`, both decrement and both save. Fixes: make the check and the update one **atomic SQL statement** — `update product set stock = stock - :qty where id = :id and stock >= :qty` with `@Modifying`, and treat "0 rows updated" as out of stock; or lock the row (`@Lock(PESSIMISTIC_WRITE)` → `SELECT ... FOR UPDATE`); or optimistic locking with retry. A Java `synchronized` block does **not** work across several instances.

```
@Modifying
@Query("update Product p set p.stock = p.stock - :qty where p.id = :id and p.stock >= :qty")
int reserve(@Param("id") long id, @Param("qty") int qty);   // returns 0 when not enough stock
```

→ *More:* §8.14 Locking: optimistic and pessimistic · §8.10 @Query: JPQL, native SQL and @Modifying

Q233. You see "HikariPool-1 - Connection is not available, request timed out after 30000ms". What are the causes and fixes? All pooled connections are in use and threads waited too long. Causes: long transactions (remote calls or slow processing inside `@Transactional`), OSIV holding connections during rendering, connection leaks (manual JDBC without closing), `REQUIRES_NEW` taking a second connection per request, slow queries, or a pool too small for the instance's concurrency (especially with virtual threads). Diagnose with the `hikaricp.connections.*` metrics, a thread dump, and `spring.datasource.hikari.leak-detection-threshold=20000` to log where a connection was borrowed. Fix the cause before enlarging the pool. → *More:* §7.2 DataSource and HikariCP · §8.15 Open Session in View

Q234. ⋆ A downstream service becomes slow and your service starts failing too (cascading failure). How do you protect it? Slow calls hold your request threads (and maybe database connections) until everything is busy. Protect with: short **timeouts** on every client; a **circuit breaker** that fails fast while the dependency is unhealthy; a **bulkhead** limiting concurrent calls to that dependency; **fallbacks** (cached data, degraded response); no remote calls inside transactions; and readiness checks that don't depend on the slow service. Monitor with per-dependency metrics. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · §6.4 Timeouts and the underlying HTTP library

Q235. ⋆ Traffic suddenly grows 10x. How would you scale a Spring Boot service? Keep the service **stateless** (sessions in Redis or tokens) so you can add instances behind a load balancer (Kubernetes HPA). Then remove bottlenecks in order: database (indexes, read replicas, connection-pool sizing — total connections across instances), caching for hot reads, moving slow work to asynchronous processing via a queue, virtual threads or WebFlux for I/O-bound concurrency, rate limiting to protect from abuse, and load tests (Gatling, k6) to find the next limit. Measure before and after each change. → *More:* §15.10 Virtual threads · §9.8 The caching abstraction

Q236. A client must import a CSV file with a million rows. How do you design it? Don't do it inside one HTTP request and one transaction. Accept the upload (stream it to storage), return **202 Accepted** with a job id, and process asynchronously: **Spring Batch** (chunk-oriented processing: read → process → write in chunks of, say, 1,000 rows with a transaction per chunk, restartable, with skip/retry policies) or a simpler `@Async`/queue worker using JDBC batch inserts. Validate per row and collect errors, report

progress through a status endpoint, and make the import idempotent so a restart doesn't duplicate rows. → *More:* §8.18 Performance checklist · §14.4 @Async and executors

Q237. An endpoint generates a report that takes two minutes. How do you expose it over REST? Use the asynchronous request–reply pattern: `POST /reports` returns **202 Accepted** with a `Location: /reports/{id}` status URL; a background worker (`@Async` on a bounded executor, or a message queue for durability across restarts) produces the report; `GET /reports/{id}` returns the status and, when done, a link to download the result. Alternatively push completion via SSE, WebSocket or a webhook. Don't hold an HTTP connection open for minutes behind load balancers with idle timeouts. → *More:* §14.4 @Async and executors · §5.4 Reading input and writing output

Q238. You must call a third-party API limited to 100 requests per second. How do you stay under the limit? Throttle on your side: a client-side rate limiter (Resilience4j `RateLimiter` or Bucket4j) around the client — shared through Redis if several instances call it; queue requests (Kafka/RabbitMQ consumer with fixed concurrency) for bursty load; honour `429` responses with `Retry-After` and exponential backoff; cache responses where allowed; and batch requests if the API supports it. → *More:* §16.8 Resilience: circuit breaker, retry, rate limiter · §6.7 Client-side API versioning and retries

Q239. A logged-in user gets 403 on POST requests while GET works, or the browser shows "blocked by CORS policy". What is wrong? 403 on state-changing requests with session or cookie authentication is almost always **CSRF**: the request lacks the CSRF token — send it from the client (`X-XSRF-TOKEN` header; `csrf(c -> c.spa())` for SPAs), or, for a pure bearer-token API, disable CSRF. A CORS error is the browser rejecting the response: configure allowed origins, methods and headers, and enable `http.cors(...)` in Spring Security so the OPTIONS preflight isn't blocked with 401 before it reaches your configuration. → *More:* §18.17 Debugging scenarios · §10.8 CSRF protection · §10.9 CORS

Q240. How do you make sure a user can only read their own orders (not someone else's by changing the id in the URL)? That bug is **IDOR/BOLA** (insecure direct object reference / broken object-level authorization), the top API security risk. Authorization must check ownership, not just "is logged in": query by owner (`findByIdAndCustomerId(id, currentUserId)` → 404 if not found), or use method security with the principal (`@PostAuthorize("returnObject.customerId == authentication.name")` or a custom permission bean `@PreAuthorize("@orderAccess.canRead(#id)")`). Write tests for the forbidden case. → *More:* §10.7 Method security

Q241. After an update, some instances keep returning old data from the cache. Why, and how do you fix it? Each instance has its own local cache (Caffeine), and `@CacheEvict` only evicts on the instance that handled the write. Fixes: use a shared cache (Redis); broadcast invalidations (Redis pub/sub, Kafka event consumed by every instance); or accept staleness with a short TTL. Also check that writes go through methods annotated with `@CachePut`/`@CacheEvict` and not around them (bulk SQL updates, other services writing the same table). → *More:* §9.8 The caching abstraction

Q242. How do you deploy a database schema change without downtime? During a rolling deployment, old and new versions run together against the same schema, so every migration must be compatible with both. Use **expand and contract**: (1) expand — add the new column/table (nullable or with a default), deploy code that writes both and reads the new; (2) backfill data in batches; (3) contract — in a later release, drop the old column once no running version uses it. Never rename or drop in one step; keep Flyway migrations small and avoid long table locks (create indexes concurrently on PostgreSQL). → *More:* §8.16 Schema migrations with Flyway · §9.4 Schema migrations: Flyway beyond the basics, and Flyway vs Liquibase

Q243. How do you build a multi-tenant Spring Boot application? Resolve the tenant per request (from the JWT claim, subdomain or header) into a request-scoped holder. Then isolate data by one of three models: **shared tables with a tenant column** (Hibernate `@TenantId` filters and sets it automatically; cheapest), **schema per tenant**, or **database per tenant** (Hibernate `MultiTenantConnectionProvider` + `CurrentTenantIdentifierResolver`, or an `AbstractRoutingDataSource`). Also partition caches, messages and rate limits by tenant; with several token issuers, configure a JWT issuer resolver per tenant. → *More:* §11.8 Multi-tenancy: several issuers in one API

Q244. Write code to retry a flaky external call three times with exponential backoff. With Framework 7 core resilience (enable with `@EnableResilientMethods` on a configuration class):

```java
@Service
class RatesClient {
    @Retryable(includes = RestClientException.class, maxRetries = 3,
               delay = 200, multiplier = 2, jitter = 50)
    public Rates fetch() {
        return restClient.get().uri("/rates").retrieve().body(Rates.class);
    }
}
```

Retry only idempotent operations and transient errors (timeouts, 503), not 4xx. For retries plus circuit breaking, use Resilience4j's `@Retry` and `@CircuitBreaker`. → *More:* §4.7 Framework 7 core resilience: @Retryable and @ConcurrencyLimit · §6.7 Client-side API versioning and retries

Q245. Write a filter that adds a correlation id to every request and to the logs.

```java
@Component
class CorrelationIdFilter extends OncePerRequestFilter {
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
            throws ServletException, IOException {
        String id = Optional.ofNullable(req.getHeader("X-Correlation-Id")).orElse(UUID.randomUUID().toString());
        MDC.put("correlationId", id);
        res.setHeader("X-Correlation-Id", id);
        try { chain.doFilter(req, res); } finally { MDC.remove("correlationId"); }
    }
}
```

`MDC` (Mapped Diagnostic Context) adds the value to every log line on that thread; always clean up in `finally` because threads are reused. With Micrometer Tracing, the `traceId` already does this — mention it. → *More:* §5.11 Filters vs interceptors (vs AOP) · §15.8 Structured logging

Q246. Write a search endpoint with optional filters, paging and sorting.

```java
@GetMapping("/api/products")
Page<ProductDto> search(@RequestParam(required = false) String name,
                        @RequestParam(required = false) BigDecimal maxPrice, Pageable pageable) {
    Specification<Product> spec = Specification.unrestricted();
    if (name != null)     spec = spec.and((r, q, cb) -> cb.like(cb.lower(r.get("name")), "%" + name.toLowerCase()
+ "%"));
    if (maxPrice != null) spec = spec.and((r, q, cb) -> cb.le(r.get("price"), maxPrice));
    return repo.findAll(spec, pageable).map(ProductDto::from);
}
```

The repository extends `JpaSpecificationExecutor<Product>`; `Pageable` binds `?page=0&size=20&sort=price,asc`. Values are bound as parameters by the Criteria API, so no SQL injection; cap the page size and whitelist sortable fields. → *More:* §8.10 @Query: JPQL, native SQL and @Modifying · §8.11 Pagination and sorting

Q247. ⋆ How would you migrate a Spring Boot 3 application to Spring Boot 4? Upgrade to the latest 3.5.x first and fix all deprecation warnings; then move to 4.x with OpenRewrite's migration recipes. Expect: renamed starters (`-web` → `-webmvc`, `-aop` → `-aspectj`, OAuth2 starters under `security-oauth2-*`), new per-module test starters, Jackson 3 packages (`tools.jackson`), `@MockBean` → `@MockitoBean`, Security 7's lambda-only DSL, Undertow gone, Spring Cloud 2025.1, and moved auto-configuration class packages. `spring-boot-starter-classic` restores the Boot 3 "everything on the classpath" behaviour as a stepping stone. Run the full test suite and compare Actuator metrics after deploying. → *More:* §17.13 Spring Boot 3.x → 4.x migration table · Ch 18 Q168

Q248. ⋆ Explain a Spring Boot project you have worked on — its architecture and your role. Structure it in about five minutes: the business problem and users; the architecture (services, data stores, messaging — draw it); the request path of one key use case end to end; the hardest technical problem you solved (with numbers: latency before/after, error rate); how it was tested, deployed and monitored; and what you would do differently. Prepare follow-ups on transactions, security, scaling and failure handling — interviewers dig into whatever you mention. → *More:* §18.16 "Explain your Spring project" — a walkthrough template

Q249. Your `@Scheduled` job sometimes runs twice at the same time or skips runs. What is going on? Three usual causes. (1) **Several instances** each run the job — add ShedLock or move it to a single scheduler. (2) **Virtual threads**: with `spring.threads.virtual.enabled=true` Boot uses `SimpleAsyncTaskScheduler`, which starts every `fixedRate` run on a new thread without waiting, so a run slower than its period overlaps the next one (the platform-thread `ThreadPoolTaskScheduler` never overlaps the same method; it just starts late runs back to back). (3) The default **single scheduler thread** is busy with another long job, so runs start late or bunch up. Also, a cron without an explicit `zone` can skip or repeat around daylight-saving changes. Make jobs idempotent, prefer `fixedDelay` for "after the previous run", and add metrics for duration and last success. → *More:* §14.7 Scheduling with @Scheduled

Q250. What does a good Spring Boot code review checklist look like? Constructor injection and no field injection; no mutable state in singletons; `@Transactional` on service methods with correct rollback and no remote calls inside; DTOs at the API boundary, validated; global exception handling with Problem Details; lazy associations fetched explicitly (no N+1, OSIV off); timeouts on clients; configuration in `@ConfigurationProperties`, no secrets in the repo; security rules tested for allowed and forbidden paths; tests at the right level (unit, slice, Testcontainers); metrics/logging for new features. → *More:* §5.13 Putting it together: design checklist and mistakes · §7.13 Transaction pitfalls · §8.18 Performance checklist

---

Sources of the question lists

Question wordings were collected (titles only — every answer above is written for this book) from:

- LeetCode Discuss — interview-experience posts for Java back-end / Spring Boot roles (search "Spring Boot interview experience", "Java backend interview experience")
- Glassdoor interview reviews for Java/Spring/microservices roles (e.g. reports of "how does `@Transactional` work under the hood", propagation levels, Kafka retries and duplicates)
- Coding Shuttle — "100 Spring Boot Interview Questions with Answers for 2 to 5 Years Experienced Java Developers"
- roadmap.sh — "Top 60+ Spring Boot Interview Questions and Answers"
- InterviewBit — "Spring Boot Interview Questions"
- GeeksforGeeks — "Spring Boot Interview Questions and Answers"
- GitHub — altafjava/spring-interview-questions-answers; Govindcoderr/Spring-boot-interview-question; rajhunter/Java-SpringBoot-Interview-Answer
- Public Notion interview-question pages (sumiteduardo "Interview Questions"; "Spring Data JPA & Hibernate — Complete Interview Notes")
- Educative — "Top 30 Spring Security Interview Questions for Java Programmers"
- Adaface — "Spring Boot interview questions"; HiPeople — "Top 50 Spring Boot Interview Questions"; GoLinuxCloud — "Spring Boot interview questions for experienced"; SecondTalent — "Spring Boot developer interview guide"
- Hibernate interview lists (Hiration, WeCreateProblems) for N+1, caches, `get` vs `load`
- Version facts checked against the Spring Boot 4.0/4.1 release notes and migration guide, the Spring Framework 7 and Spring Security 7 reference documentation, and the Hibernate ORM 7.0 migration guide